ECE0202 Embedded Processors & Interfacing + Lab

Sichuan University-Pittsburgh Institute

Interrupt *vs* Polling

Polling:

You pick up the phone every few seconds to check whether you are getting a call.

Interrupt *vs* Polling

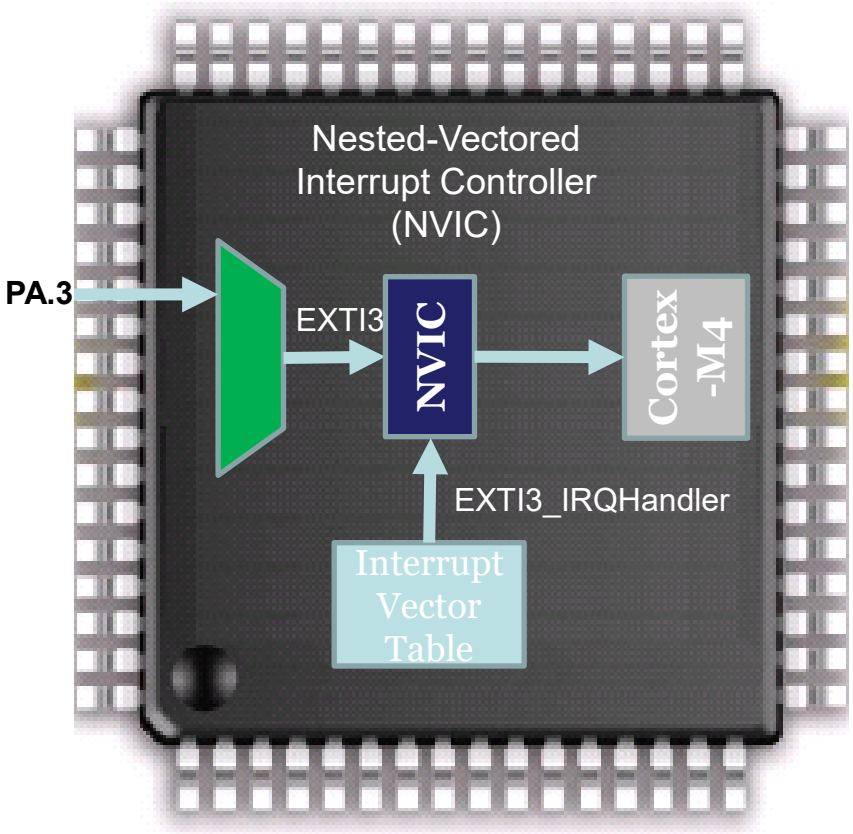
http://www.istockphoto.com/

Interrupt:

Do whatever you should do and pick up the phone when it rings.

- Hardware-triggered software action
- Respond to external events efficiently, while avoiding busy wait.
- When no events, the processor can run the main program or enter a sleep state to conserve energy

Interrupt Vector Table



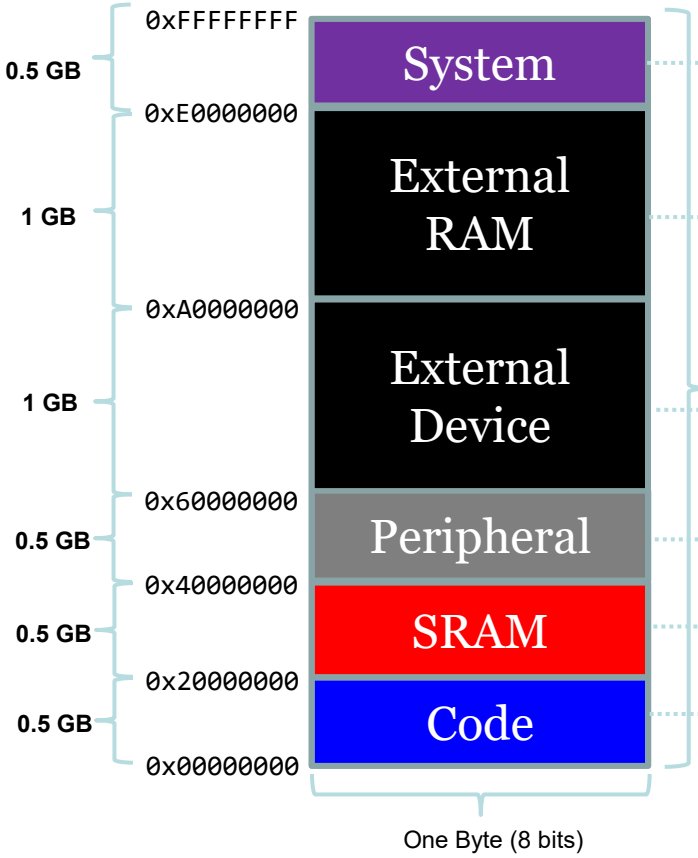
Address of ISR 1

Interrupt Vector Table

Interrupt Number (8 bits)	Memory Address of ISR (32 bits)
1	Interrupt Service Routine for interrupt 1
2	Interrupt Service Routine for interrupt 2
3	Interrupt Service Routine for interrupt 3
4	Interrupt Service Routine for interrupt 4
5	Interrupt Service Routine for interrupt 5
…	…

When interrupt x is triggered, jump to the ISR for interrupt x. ($1 \leq x \leq 255$)

Memory Map of Cortex-M4



NVIC, System Timer, SCB, vendor-specific memory

Off-chip memory for data

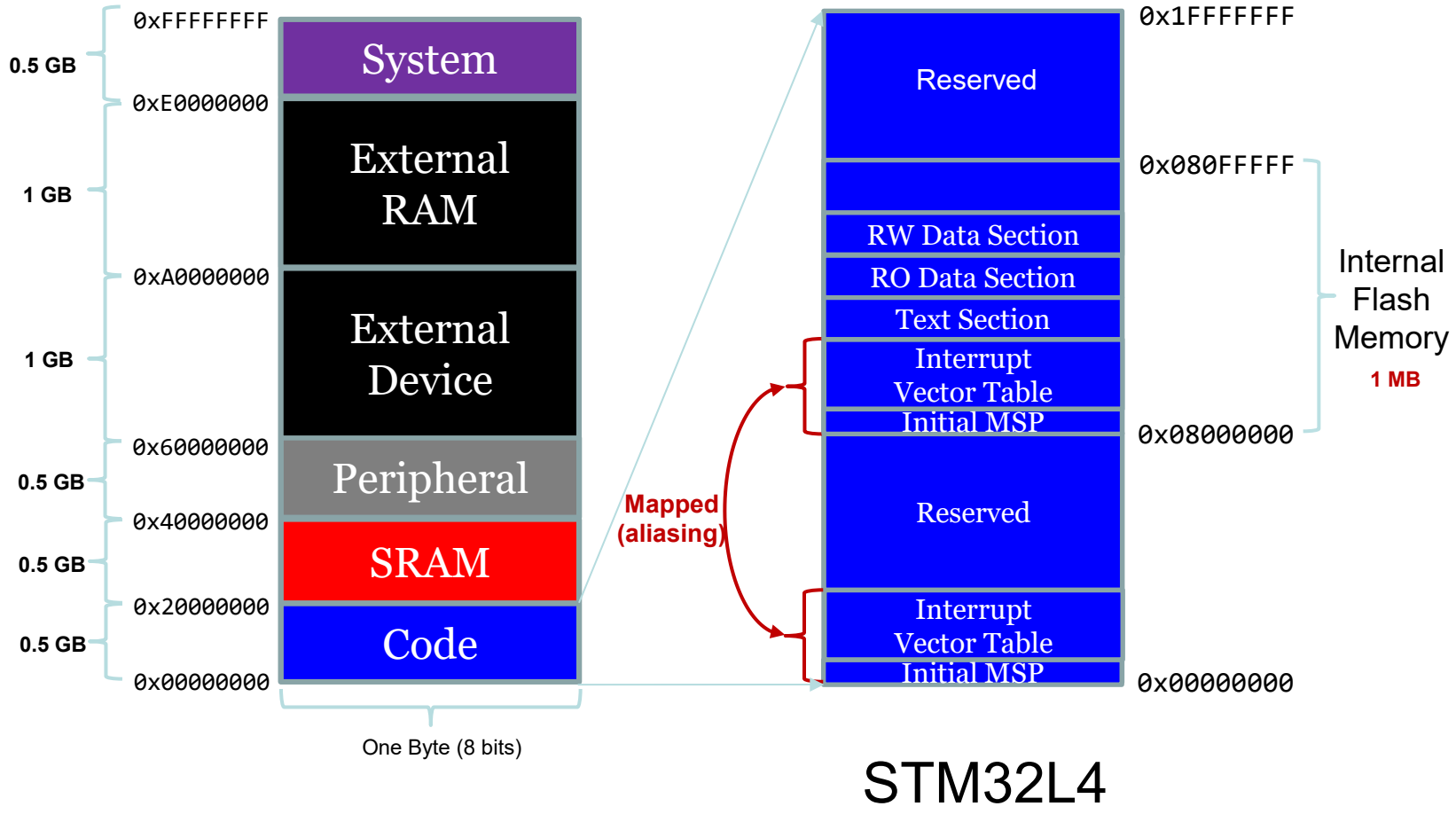
Such as SD card

AHB & APB, such as timers, GPIO

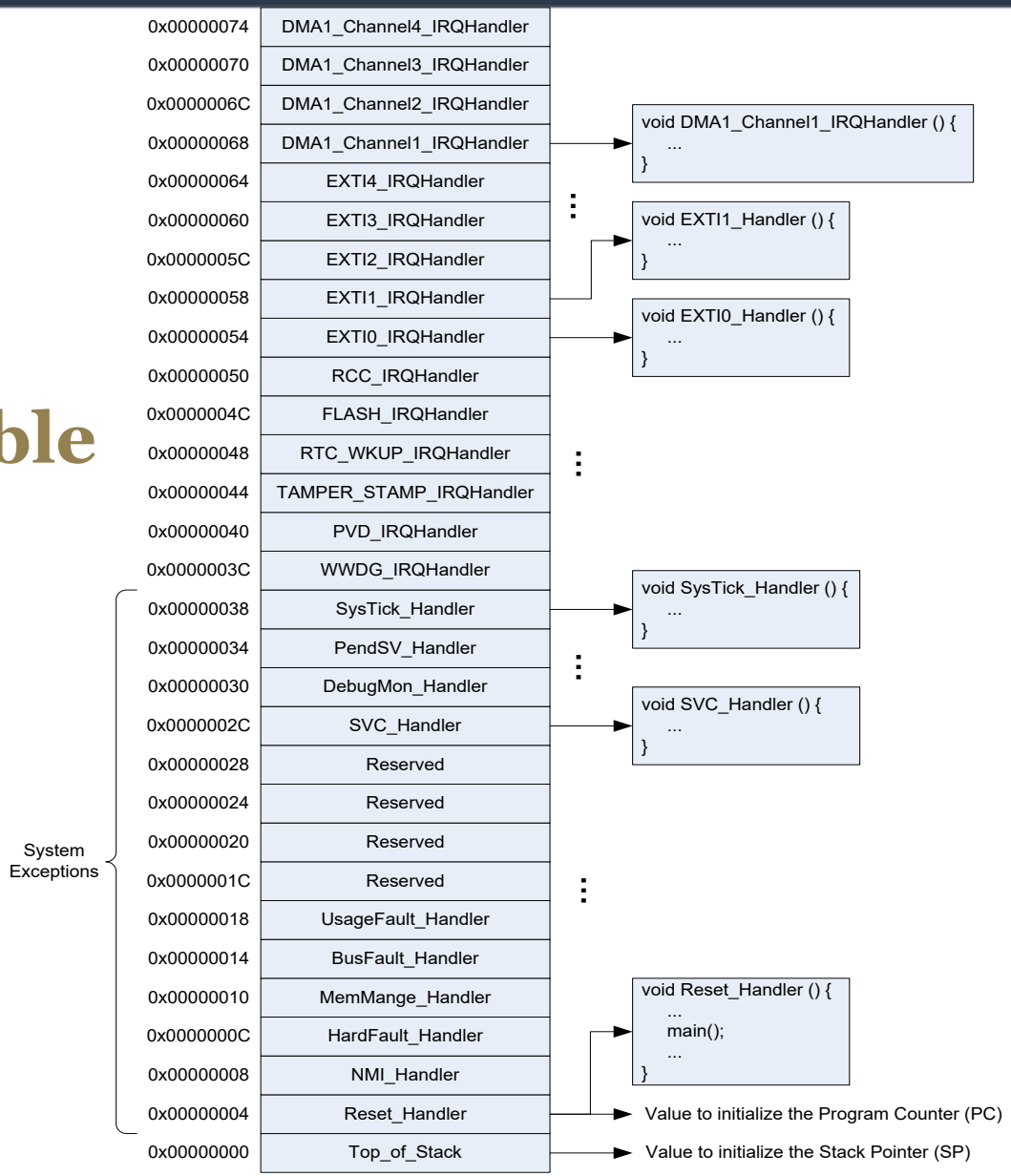
On-chip RAM, for heap, stack, & code

On-chip Flash, for code & data

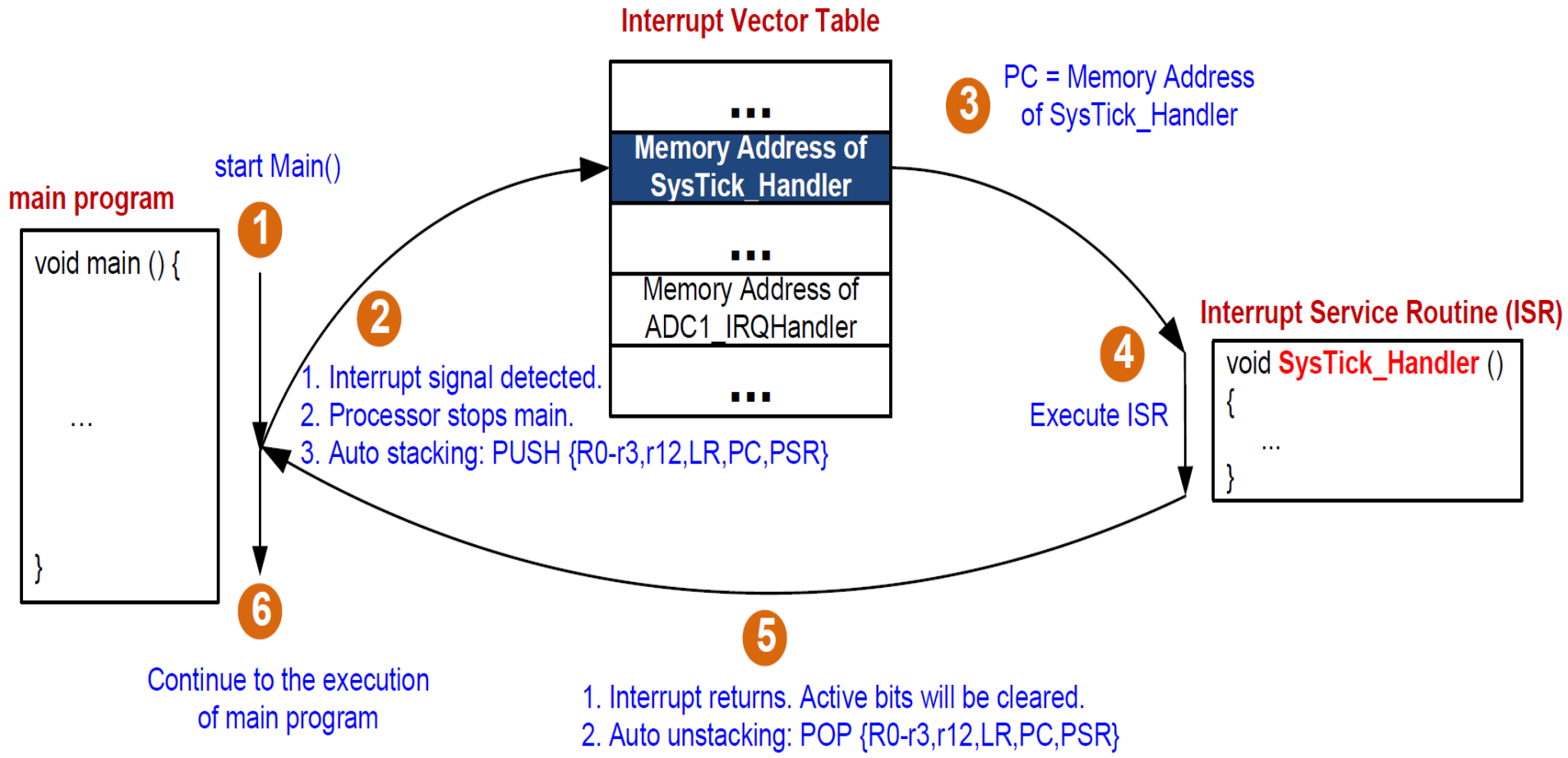
Instruction Memory



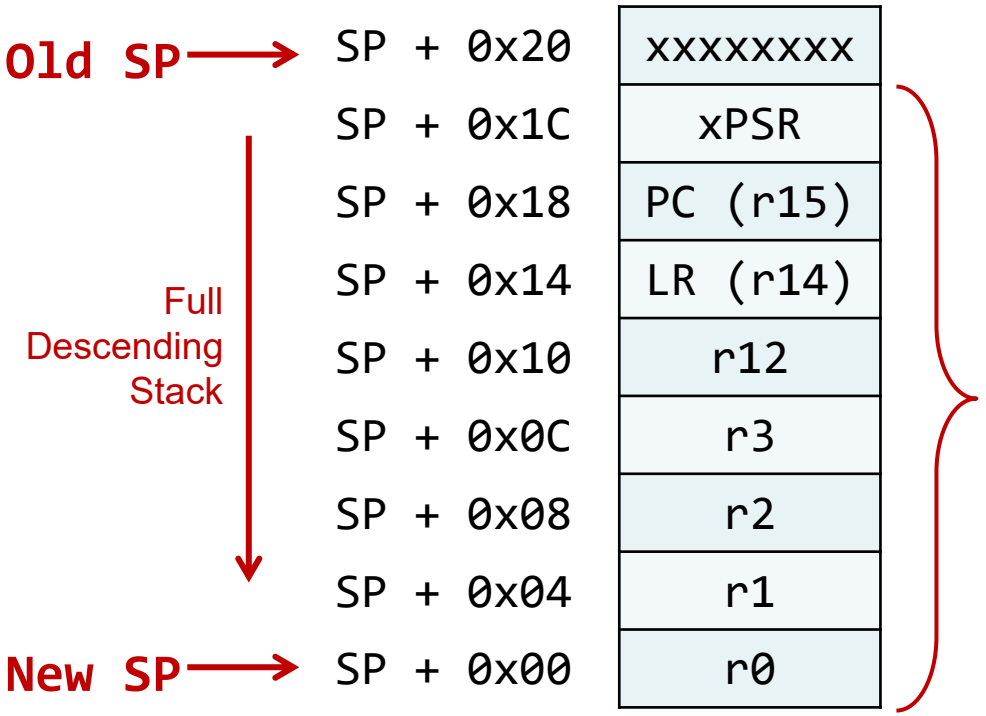
ISR Vector Table



Interrupt

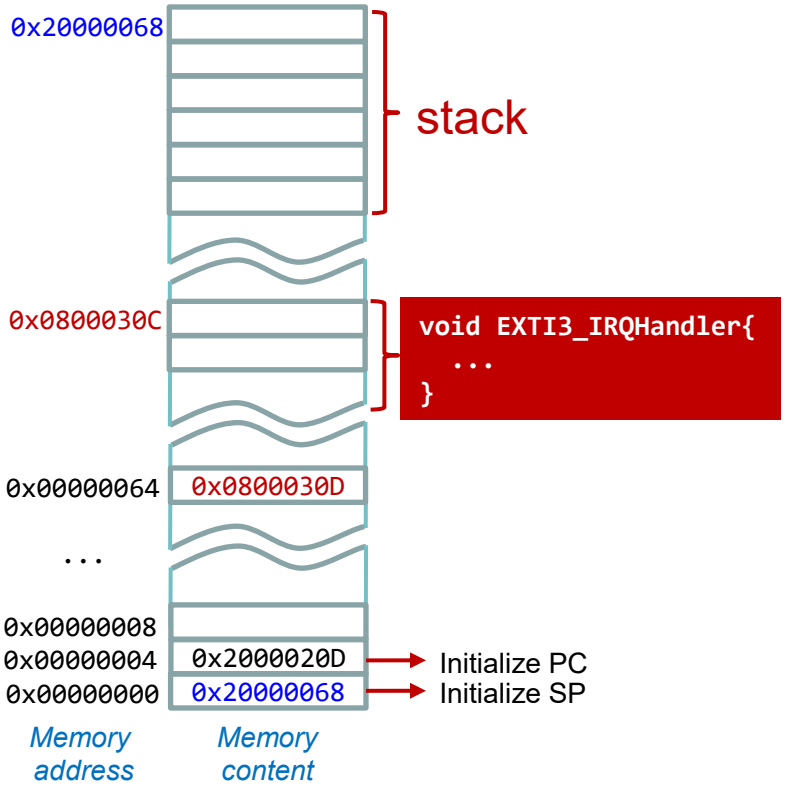

Interrupt Vector Table
...
Memory Address of SysTick_Handler
...
Memory Address of ADC1_IRQHandler
...
3
PC = Memory Address of SysTick_Handler
start Main()
main program
1
void main () {
...
}
2
1. Interrupt signal detected.
2. Processor stops main.
3. Auto stacking: PUSH {R0-r3,r12,LR,PC,PSR}
Interrupt Service Routine (ISR)
4
Execute ISR
void SysTick_Handler ()
{
...
}
5
1. Interrupt returns. Active bits will be cleared.
2. Auto unstacking: POP {R0-r3,r12,LR,PC,PSR}
6
Continue to the execution of main program

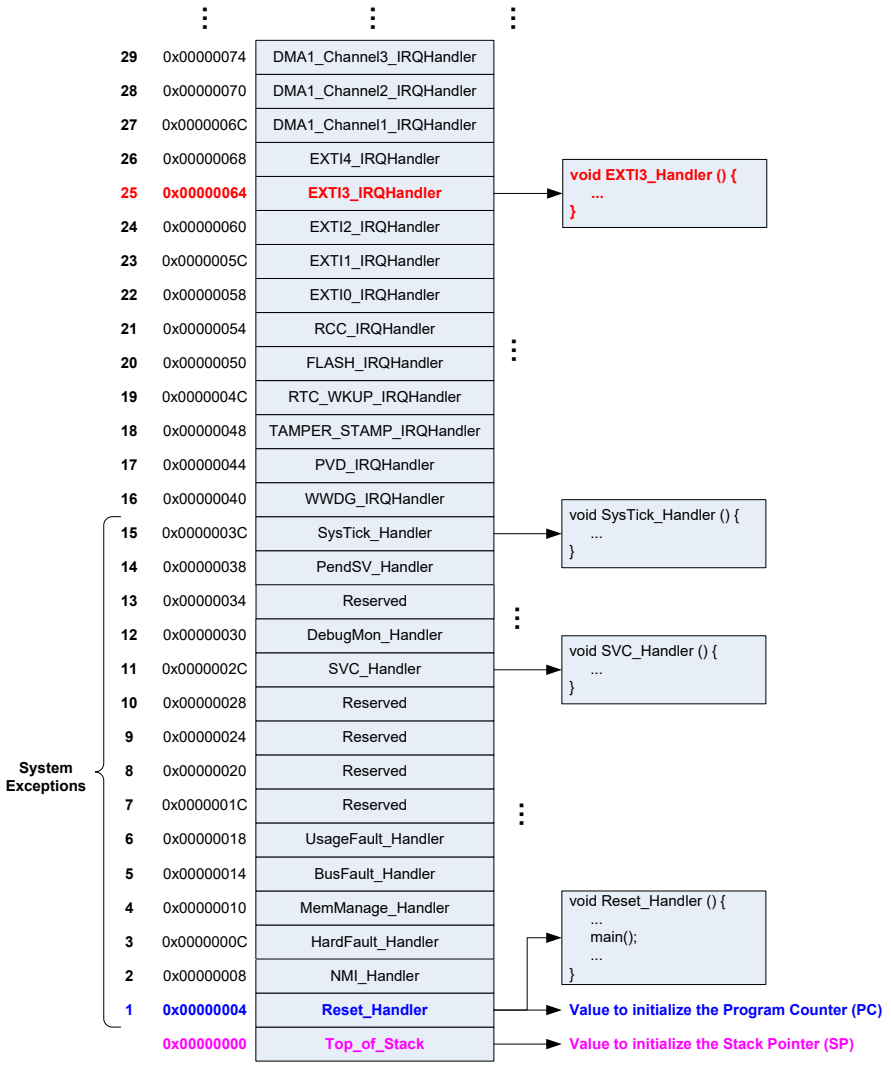
Stacking & Unstacking



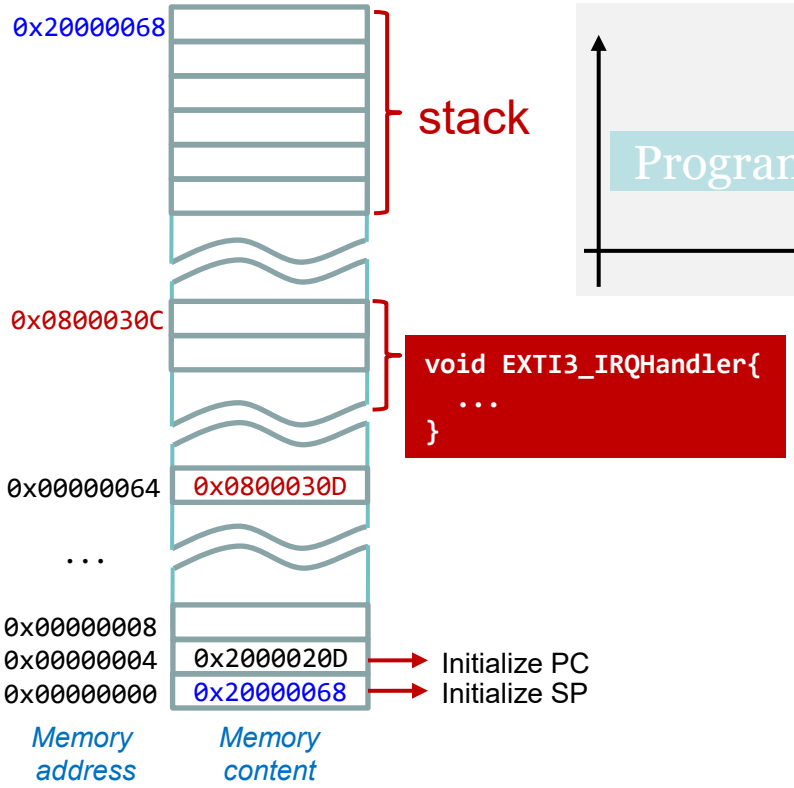
- **Stacking**: The processor automatically pushes these eight registers into the main stack before an interrupt handler **starts**
- **Unstacking:** The processor automatically pops these eight registers out of the main stack when an interrupt hander **exits**.

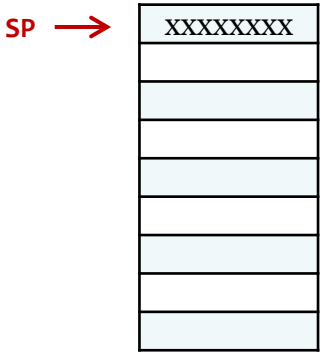
Interrupt Service Routine (ISR)





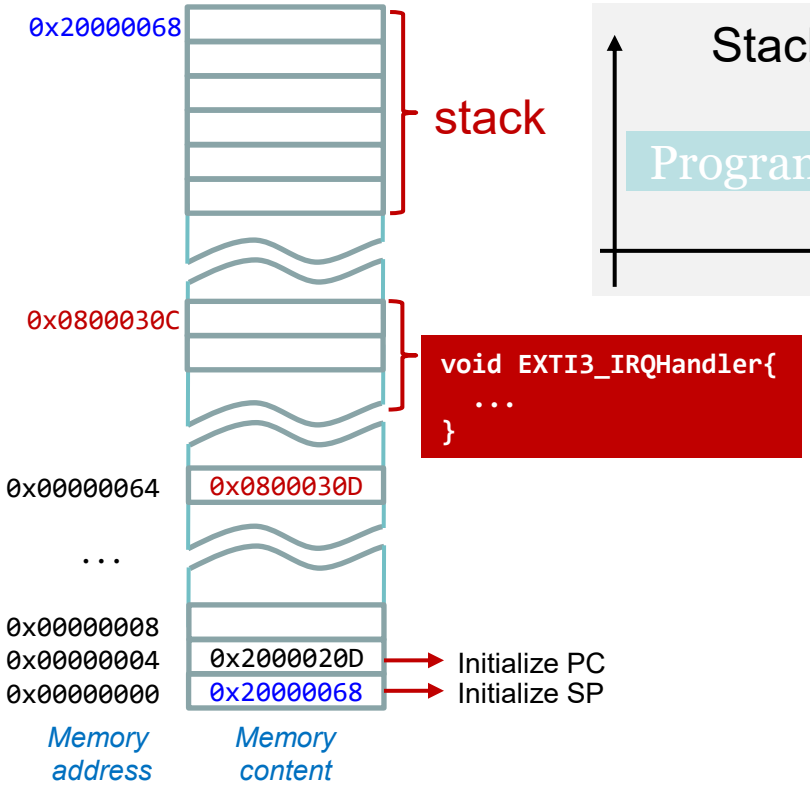
Single Interrupt

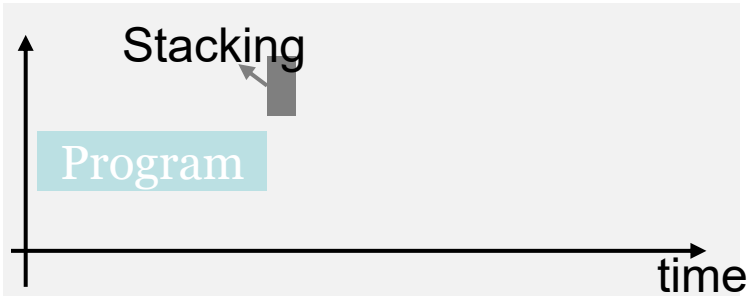


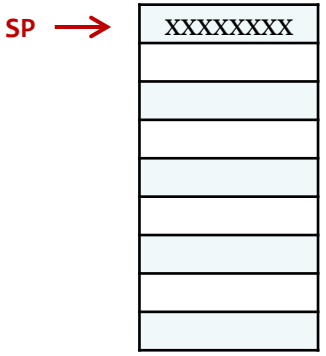


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

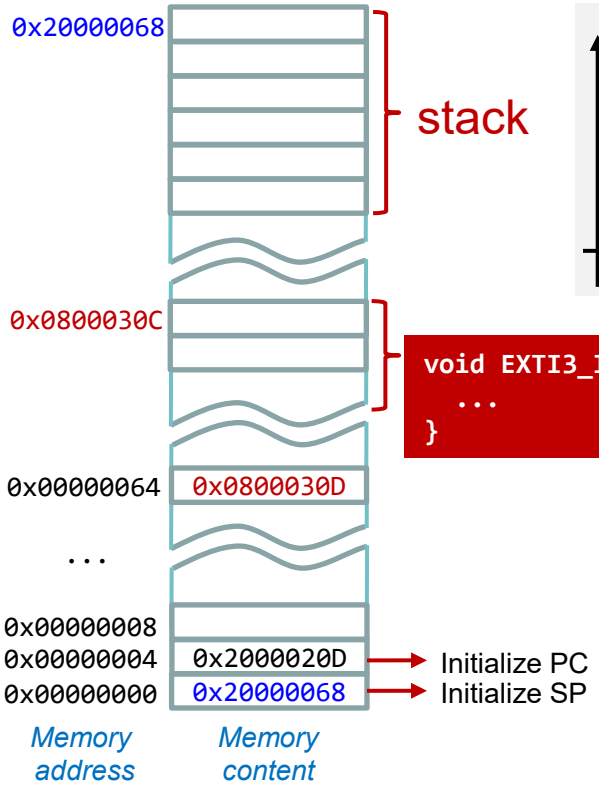


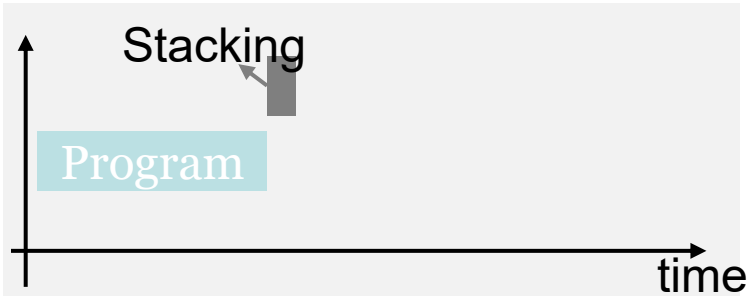


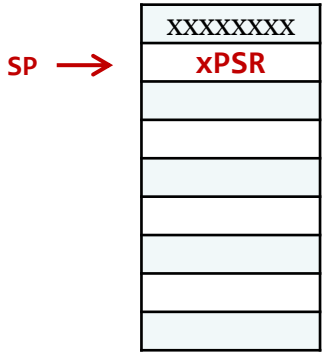


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	1	0
Priority Register	3	4	7	2	8

Single Interrupt

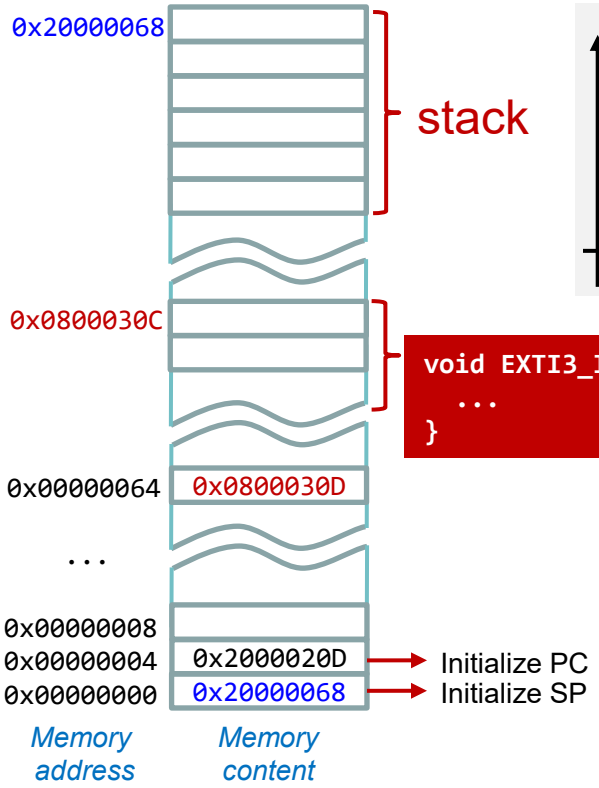


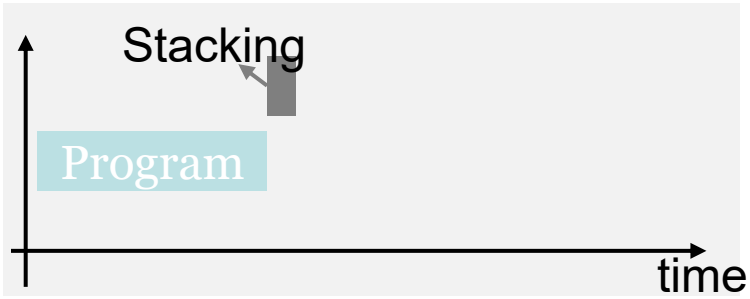


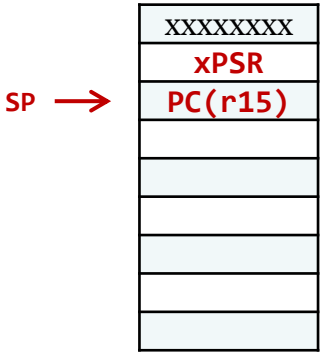


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	1	0
Priority Register	3	4	7	2	8

Single Interrupt

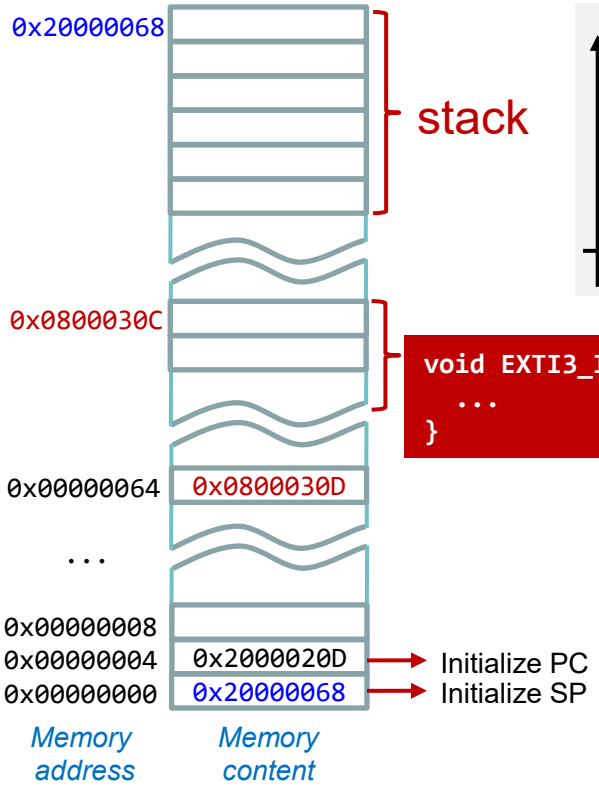


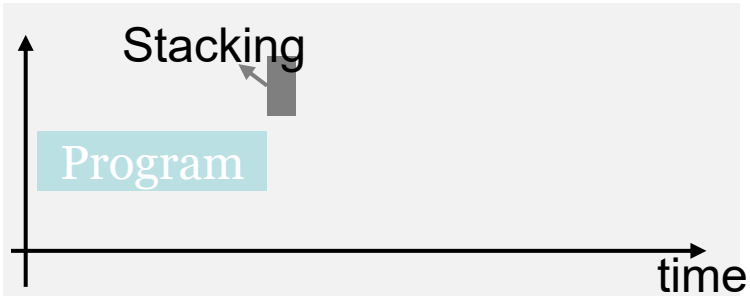


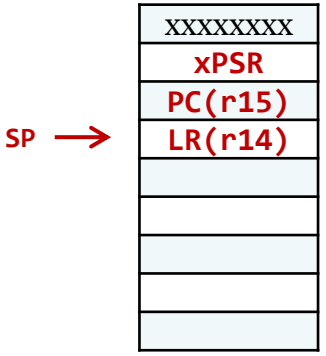


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	1	0
Priority Register	3	4	7	2	8

Single Interrupt

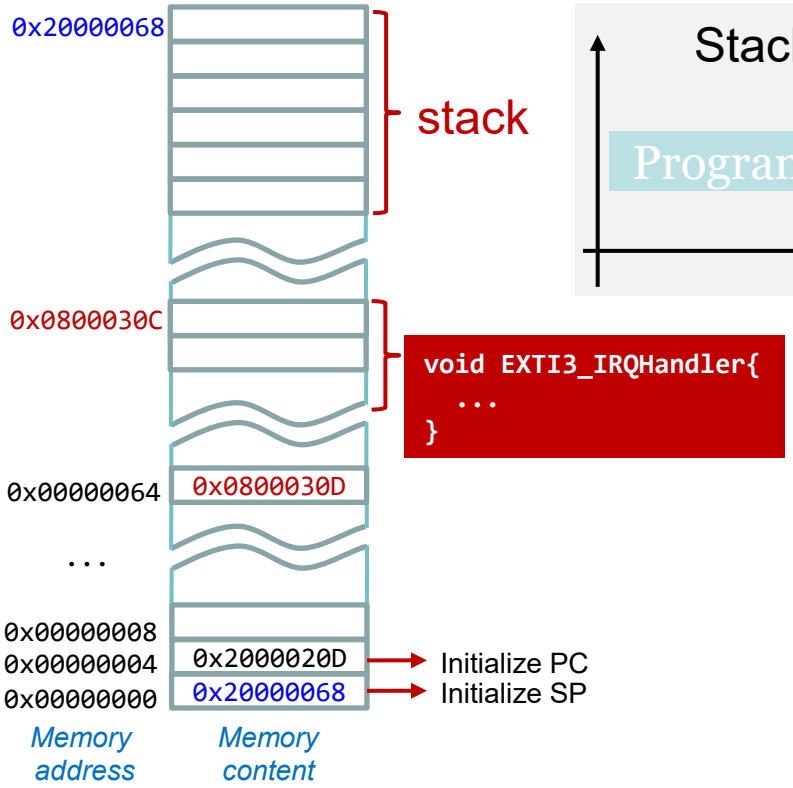


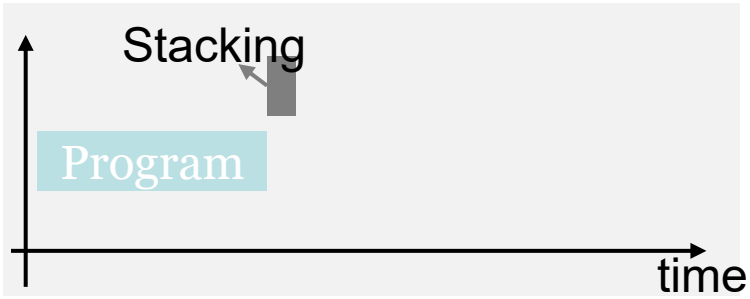


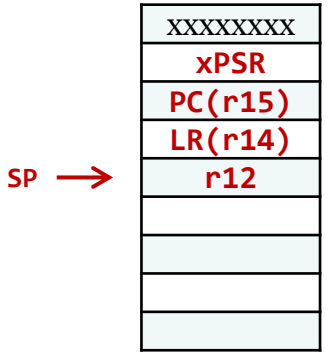


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	1	0
Priority Register	3	4	7	2	8

Single Interrupt

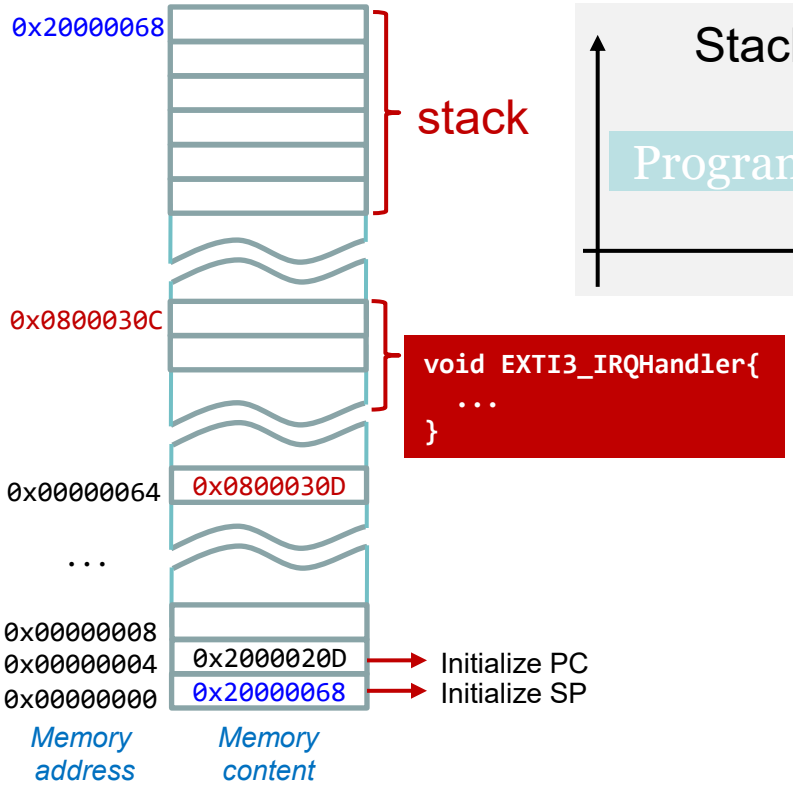


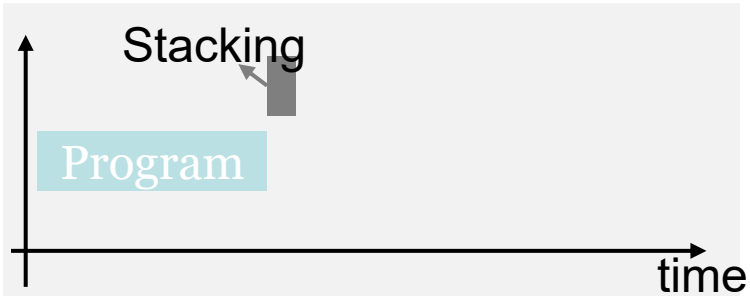


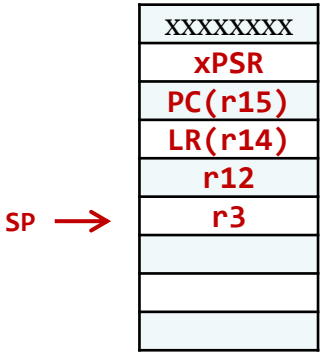


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	1	0
Priority Register	3	4	7	2	8

Single Interrupt

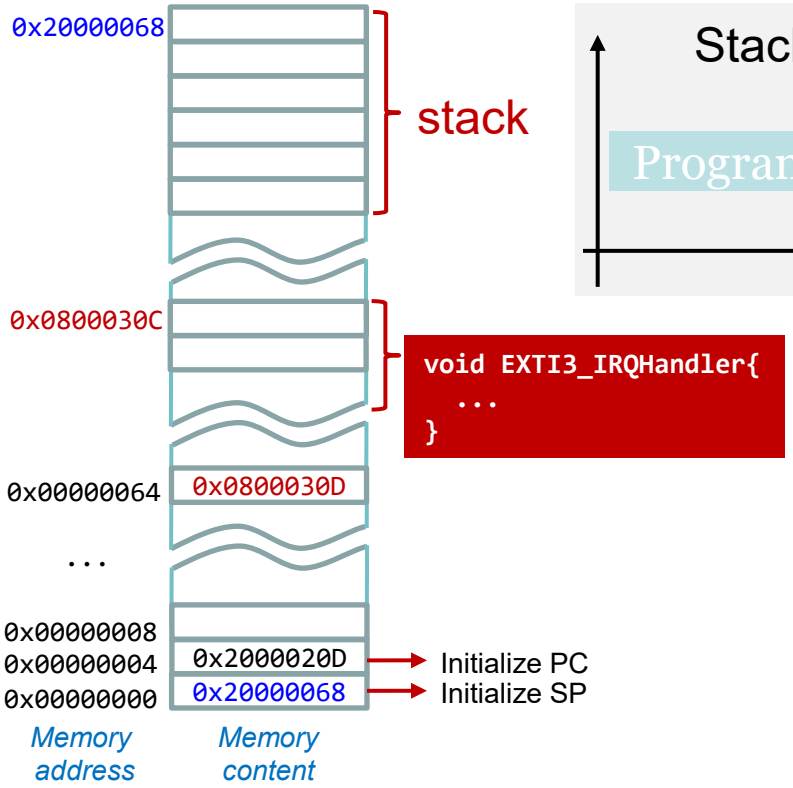


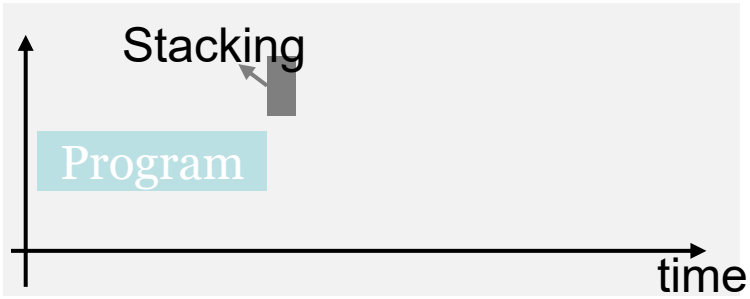


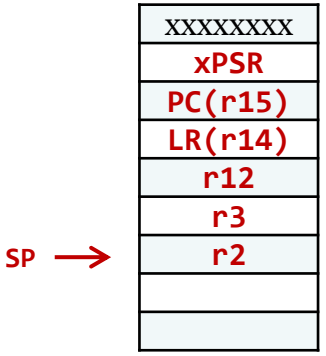


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	1	0
Priority Register	3	4	7	2	8

Single Interrupt

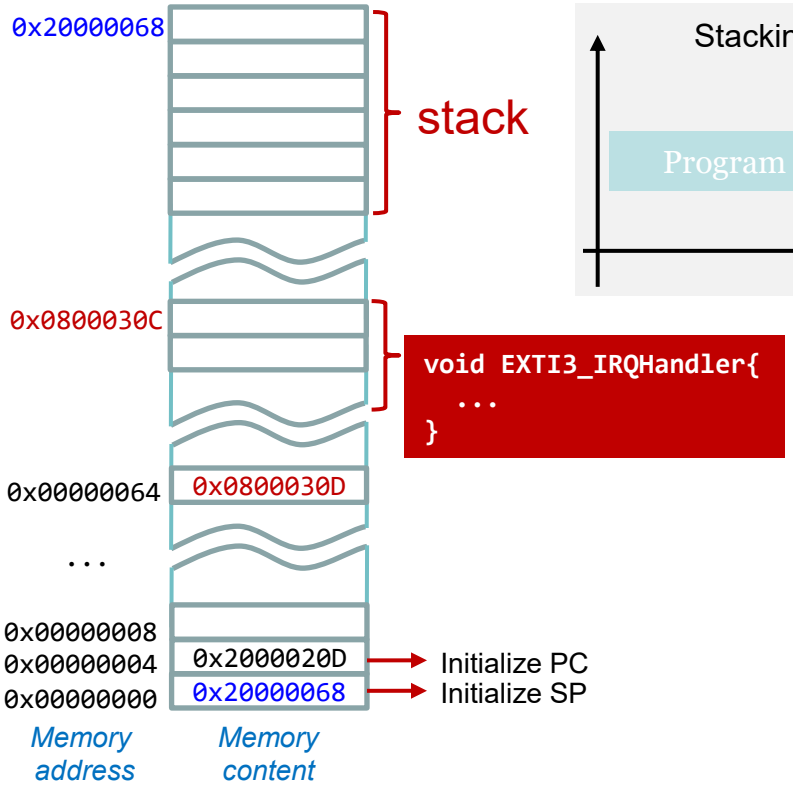


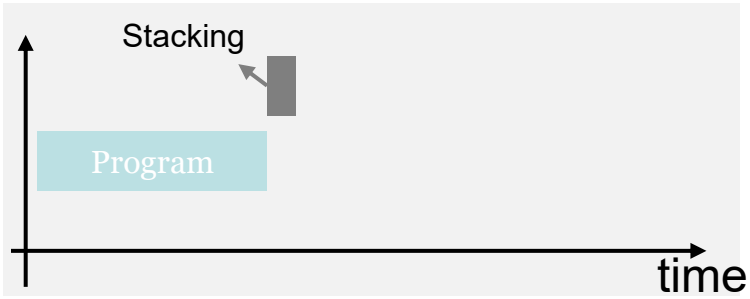


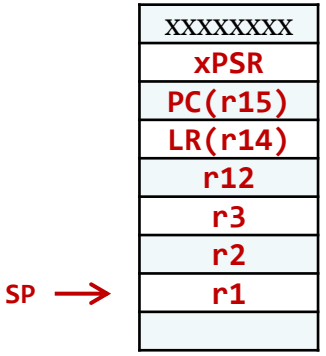


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	1	0
Priority Register	3	4	7	2	8

Single Interrupt

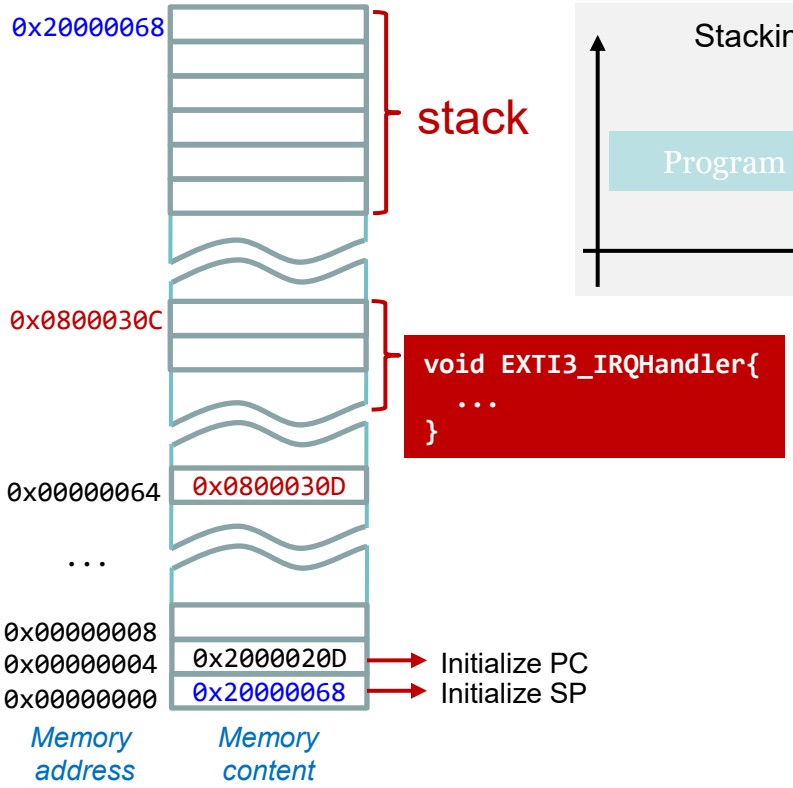


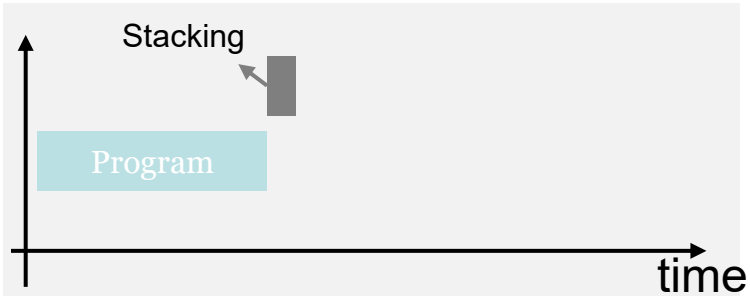


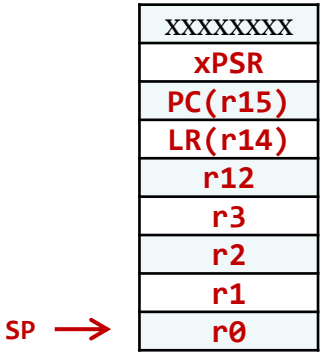


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	1	0
Priority Register	3	4	7	2	8

Single Interrupt

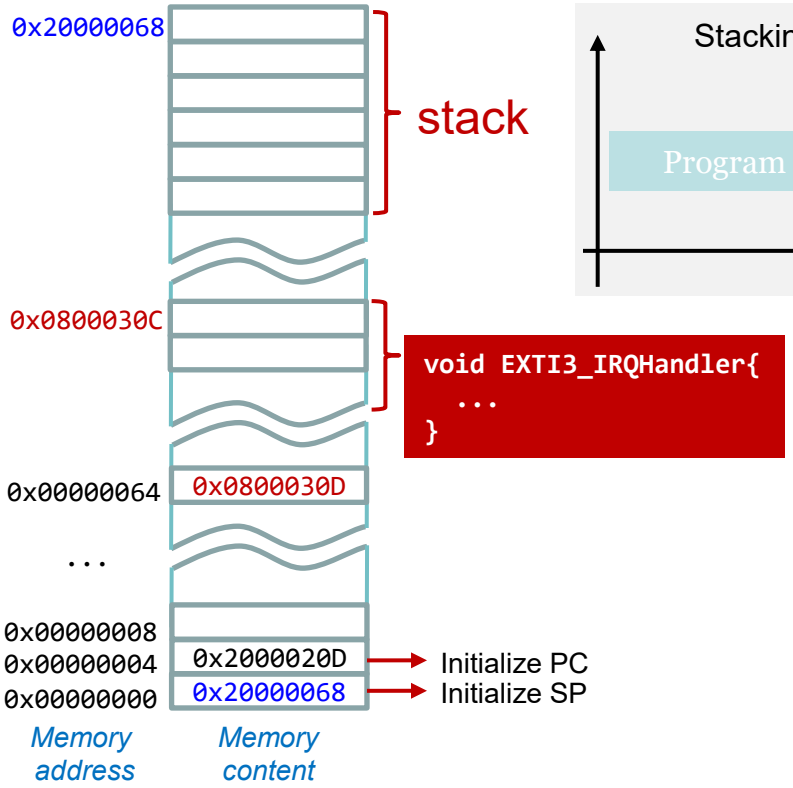


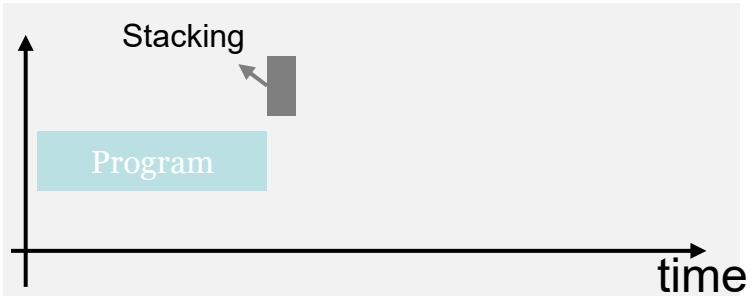


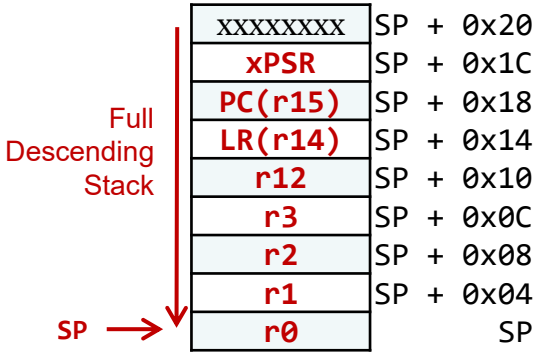


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	1	0
Priority Register	3	4	7	2	8

Single Interrupt

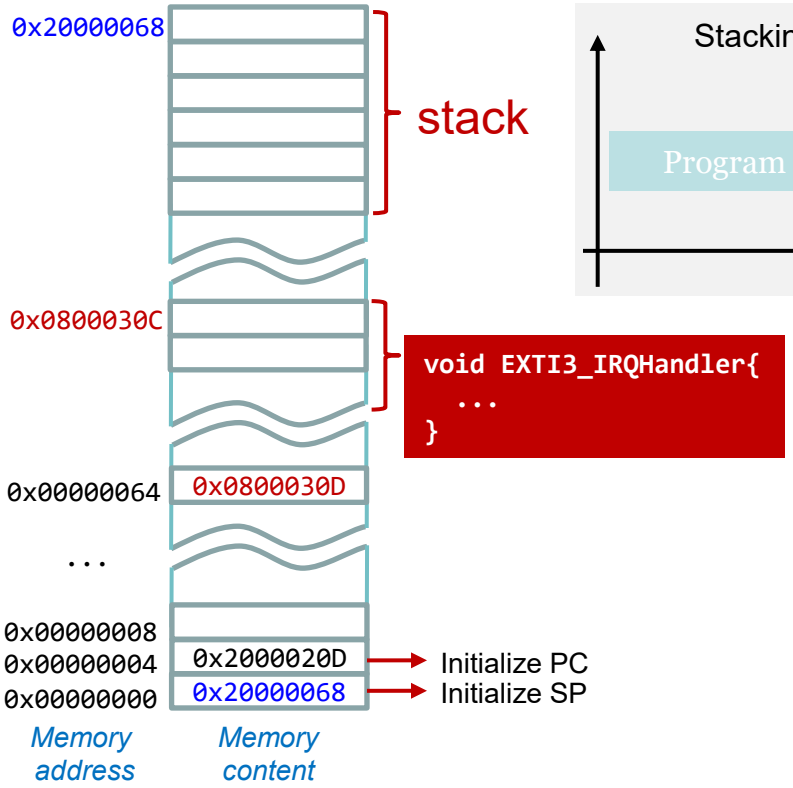


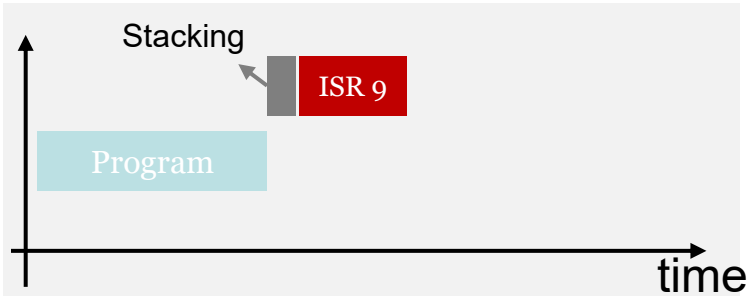


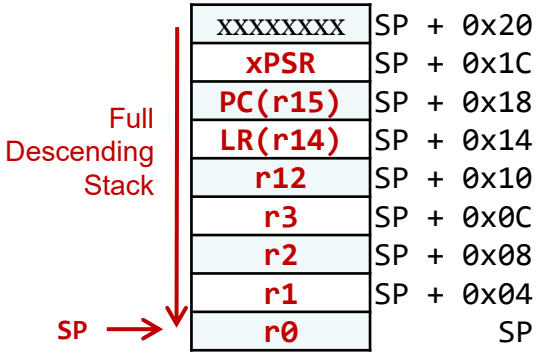


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	1	0
Priority Register	3	4	7	2	8

Single Interrupt

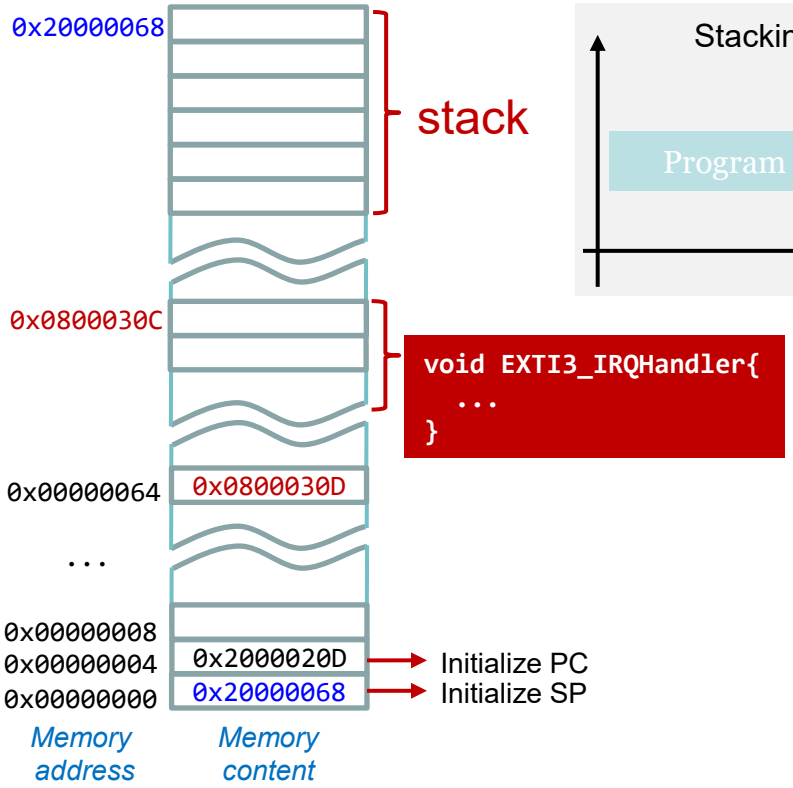


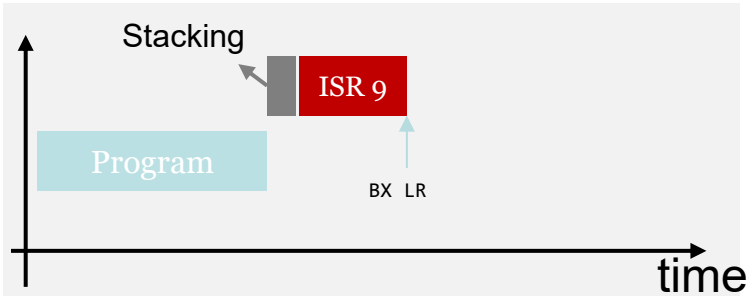


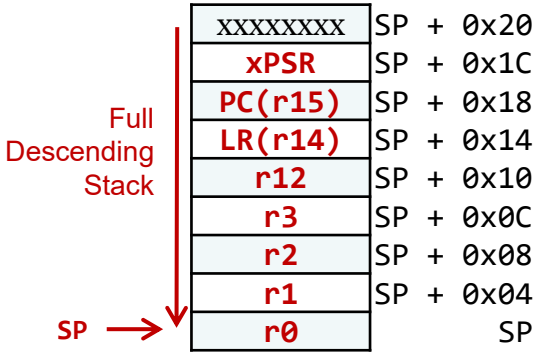


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	1	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

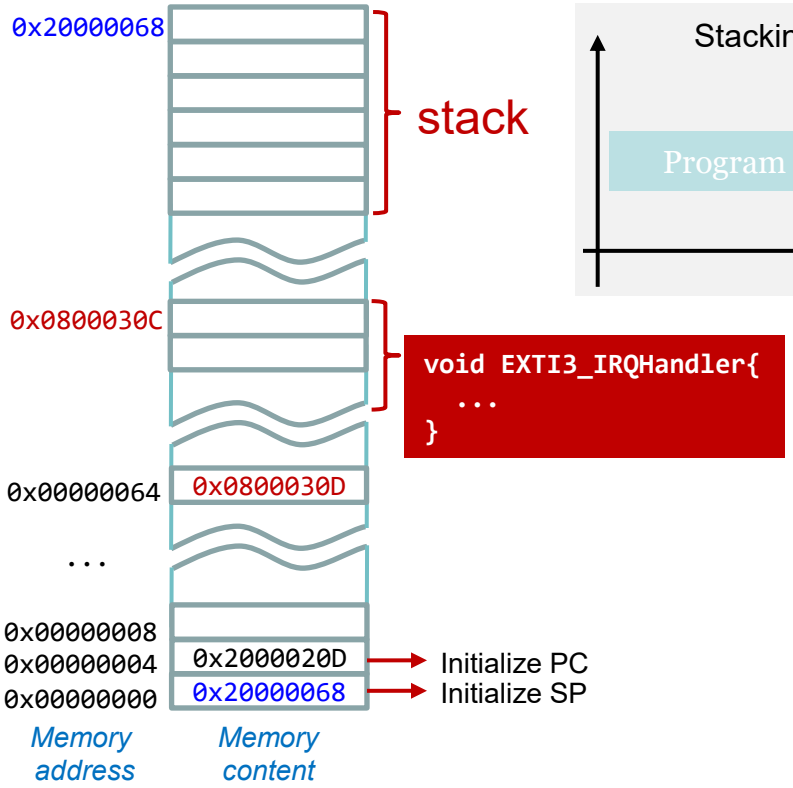


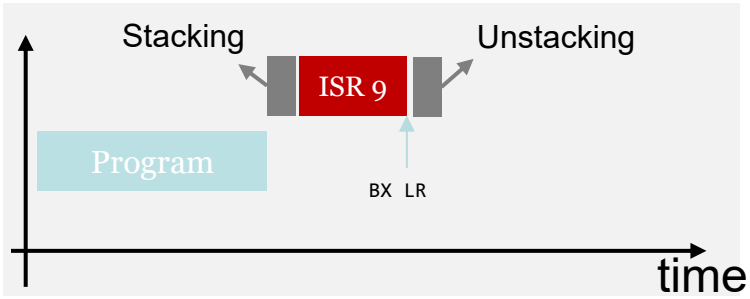


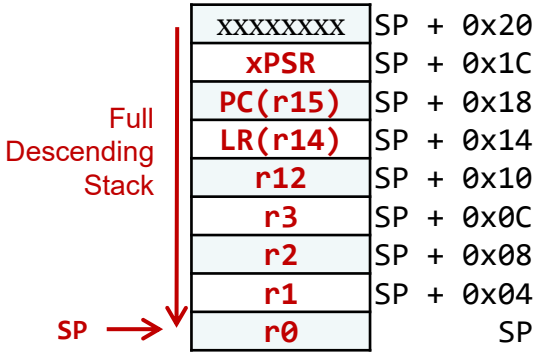


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	1	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

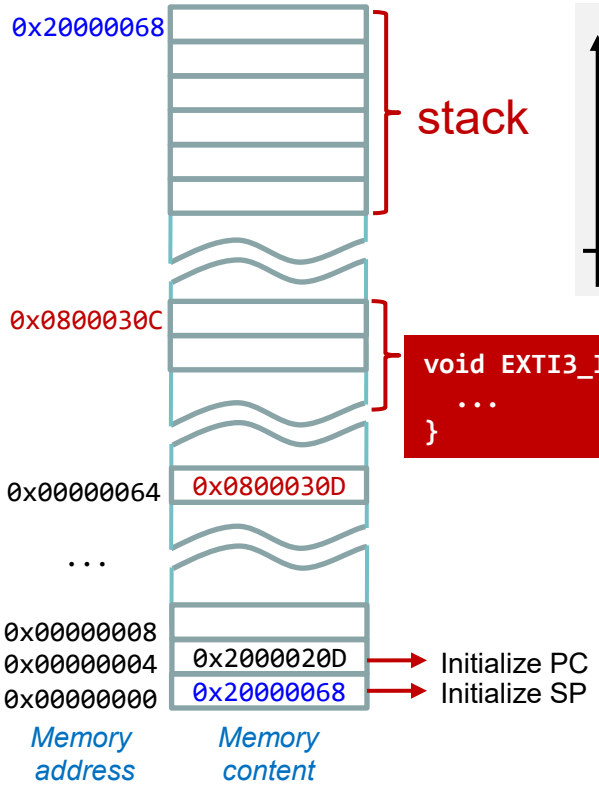


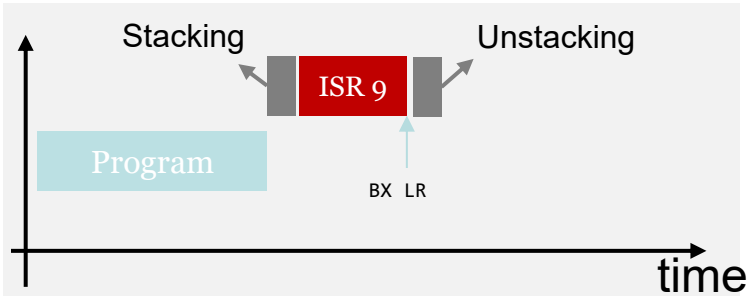


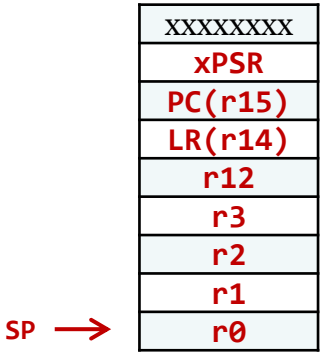


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

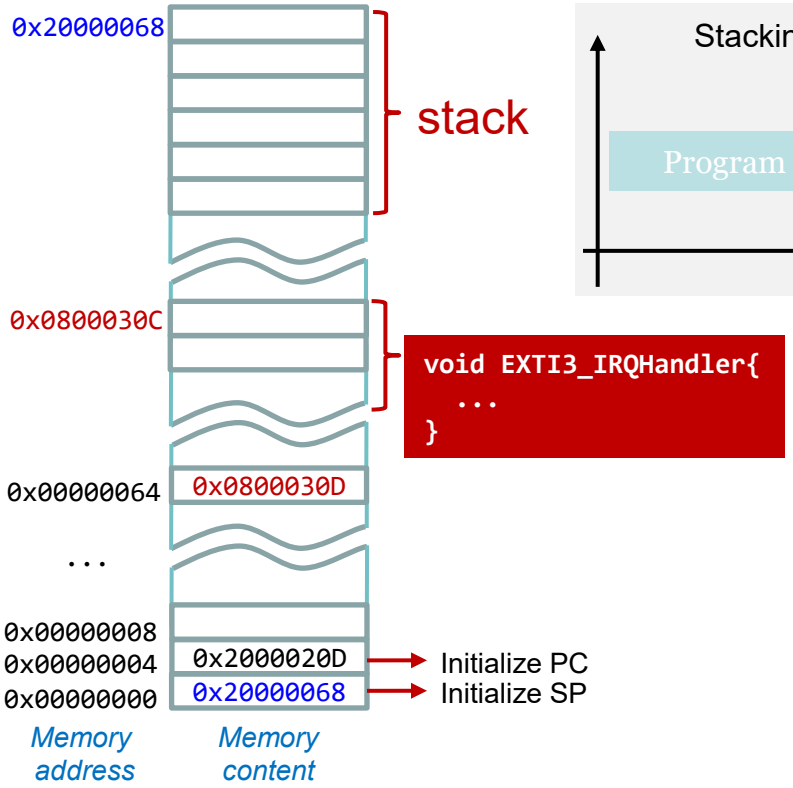


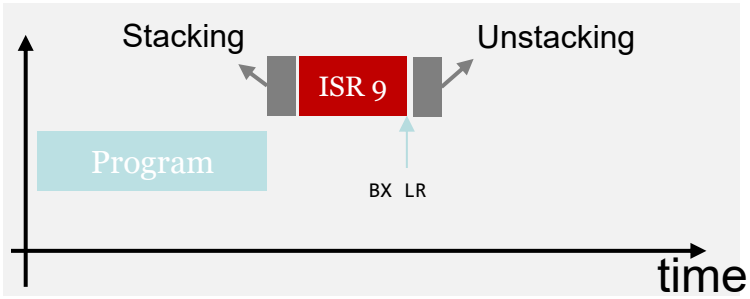


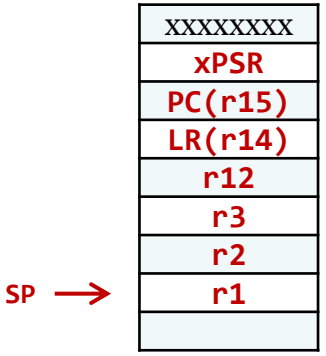


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

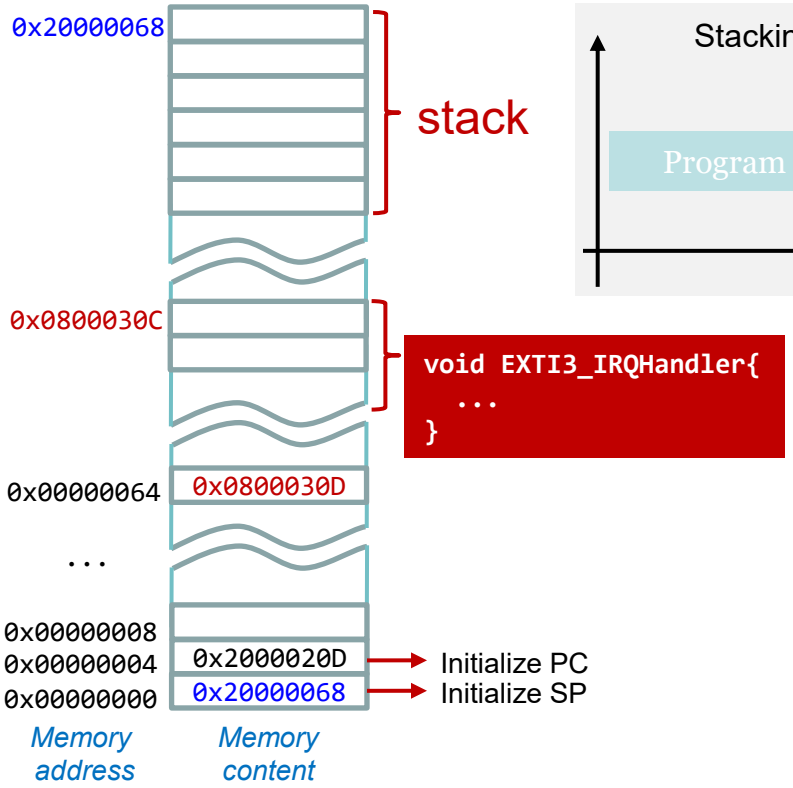


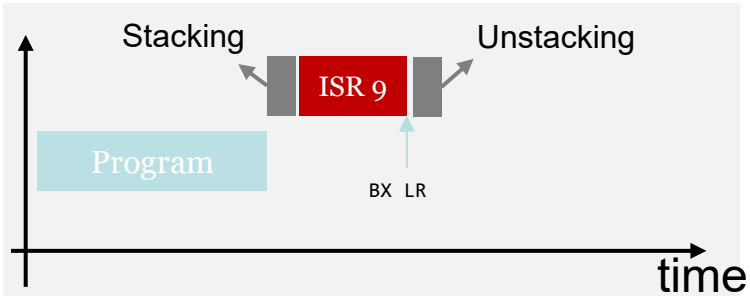


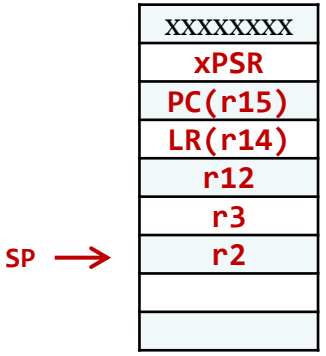


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

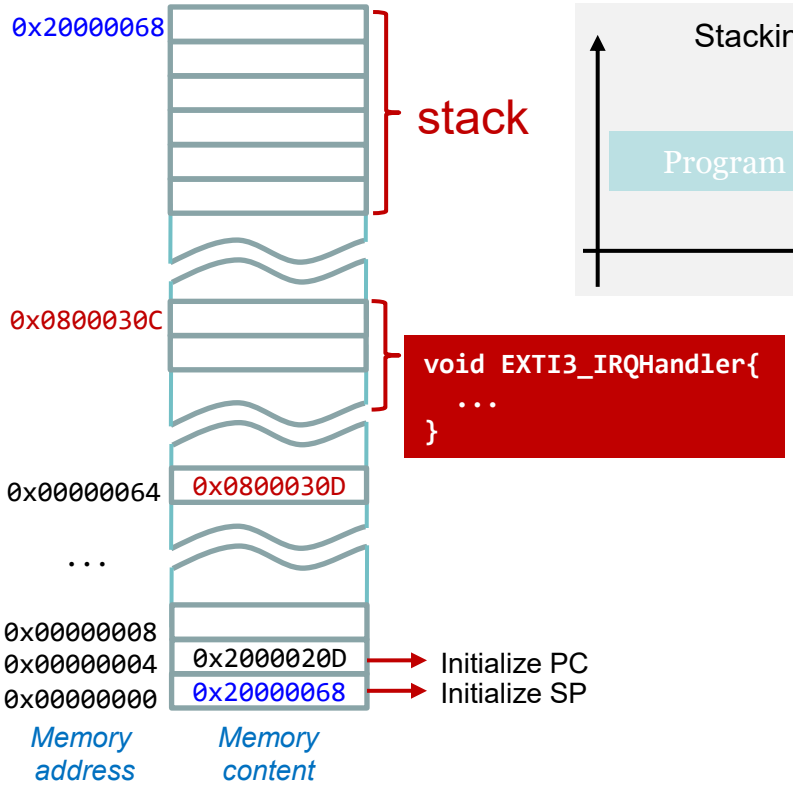


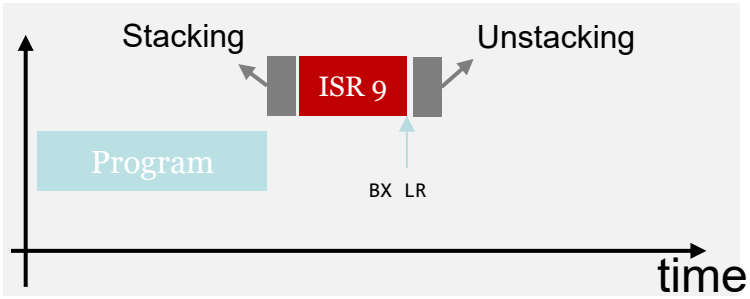


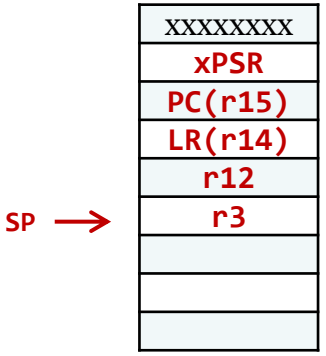


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

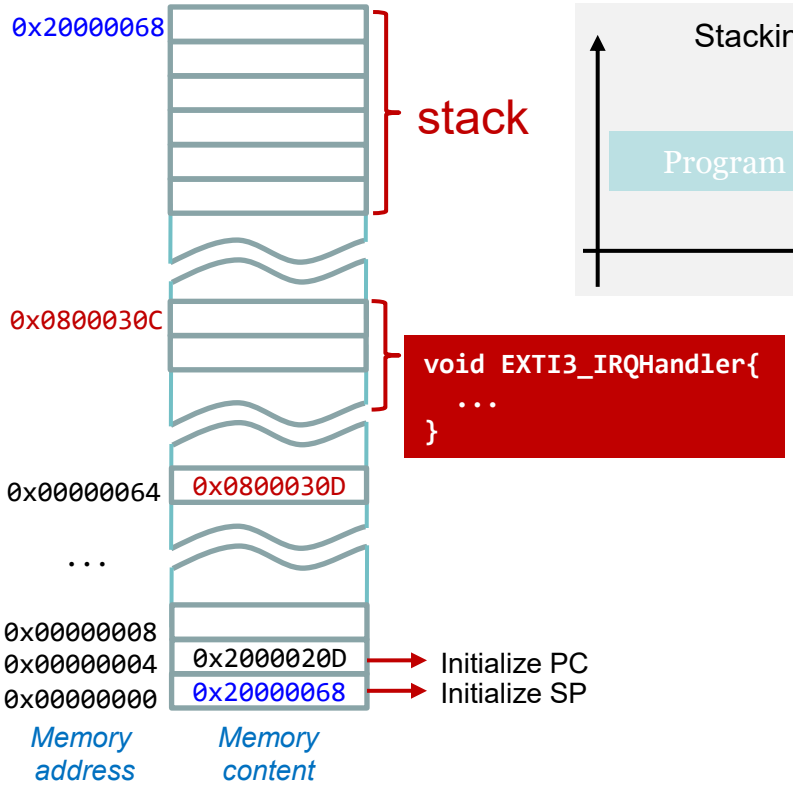


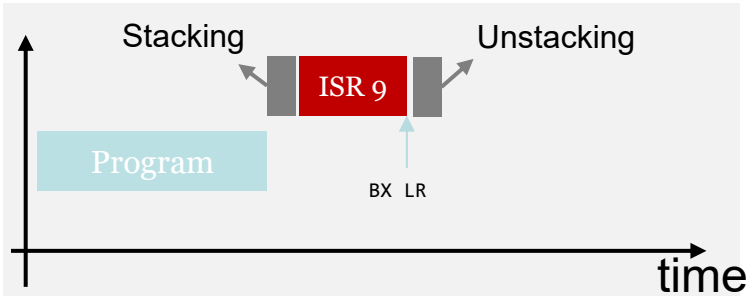


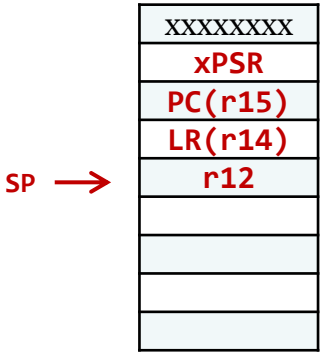


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

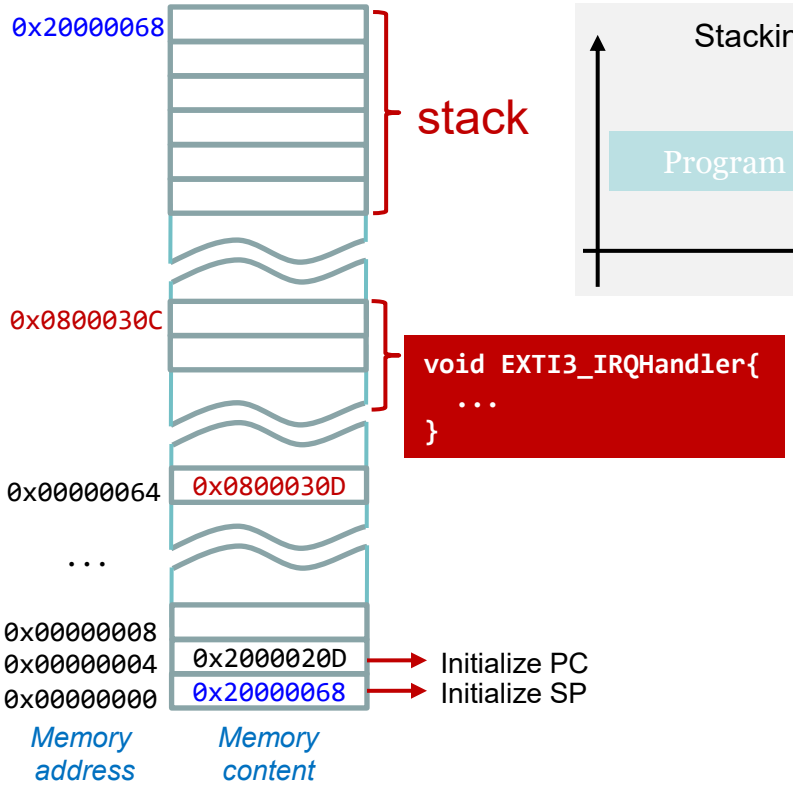


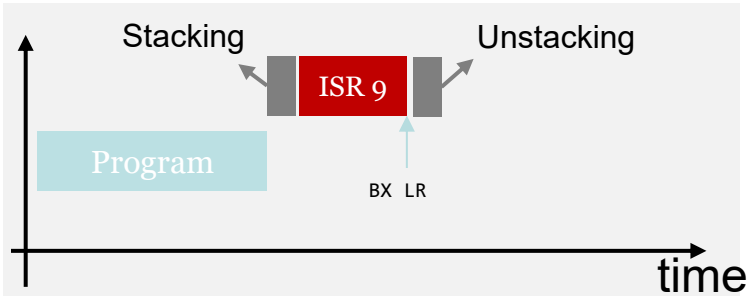


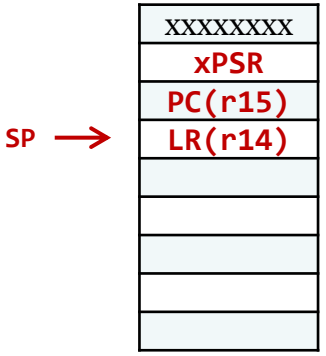


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

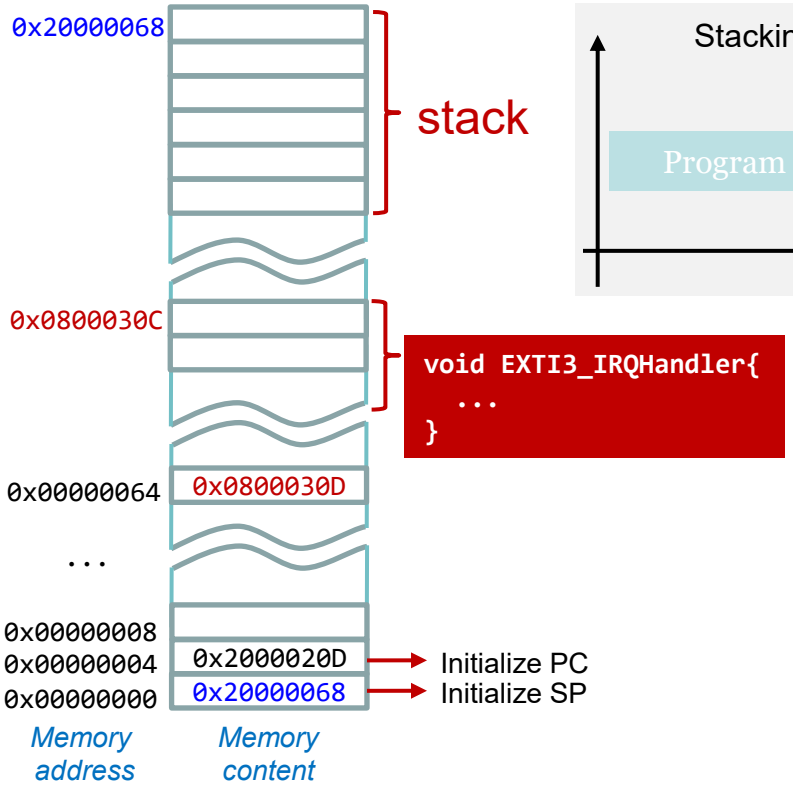


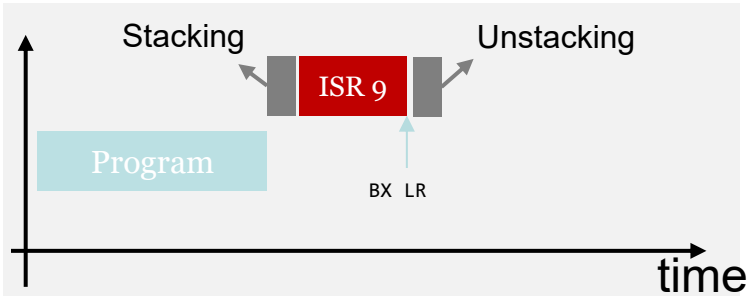


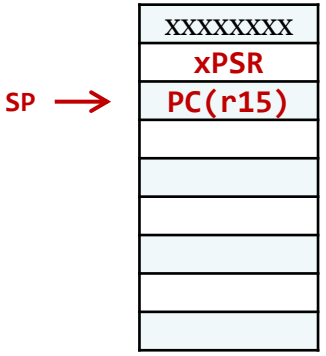


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

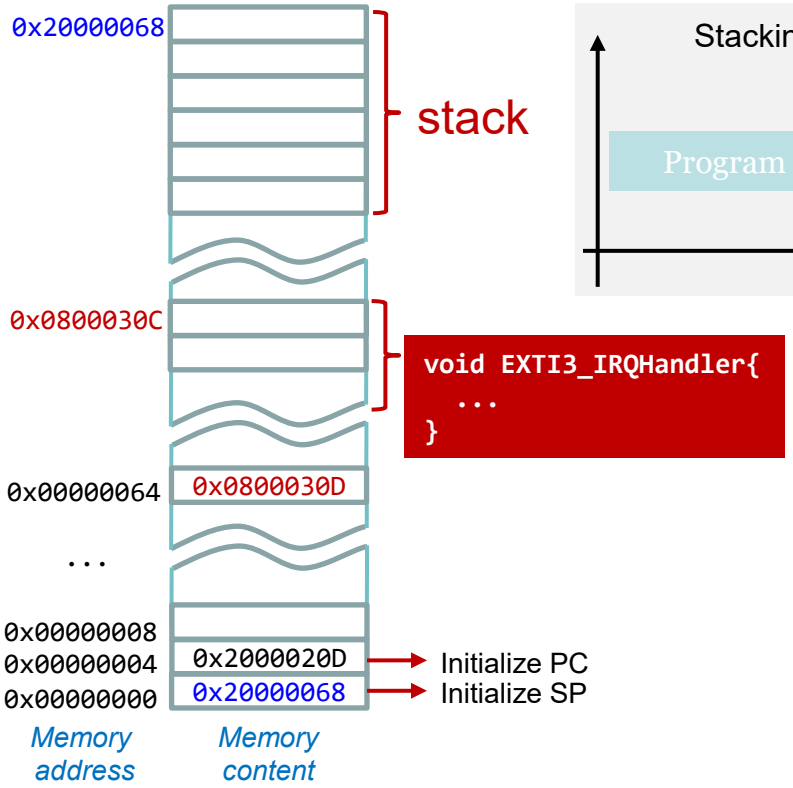


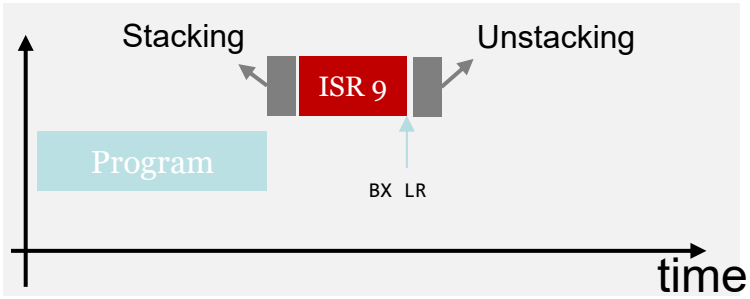


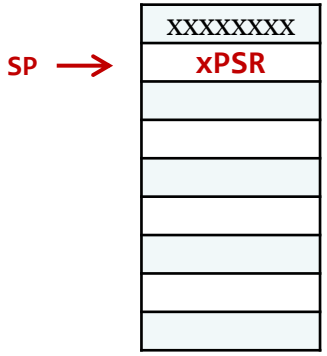


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

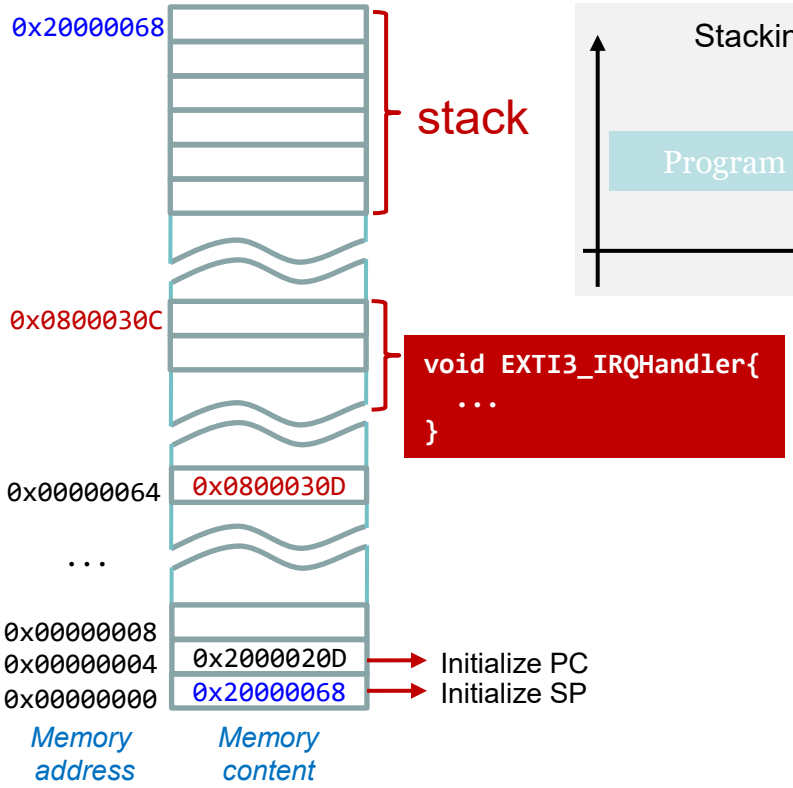


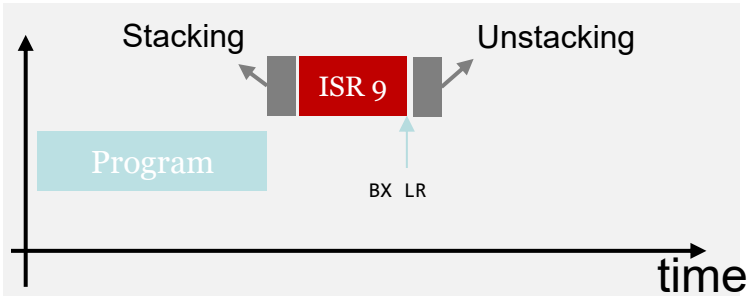


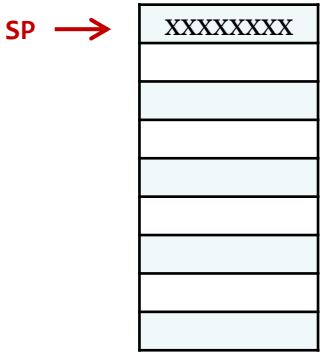


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

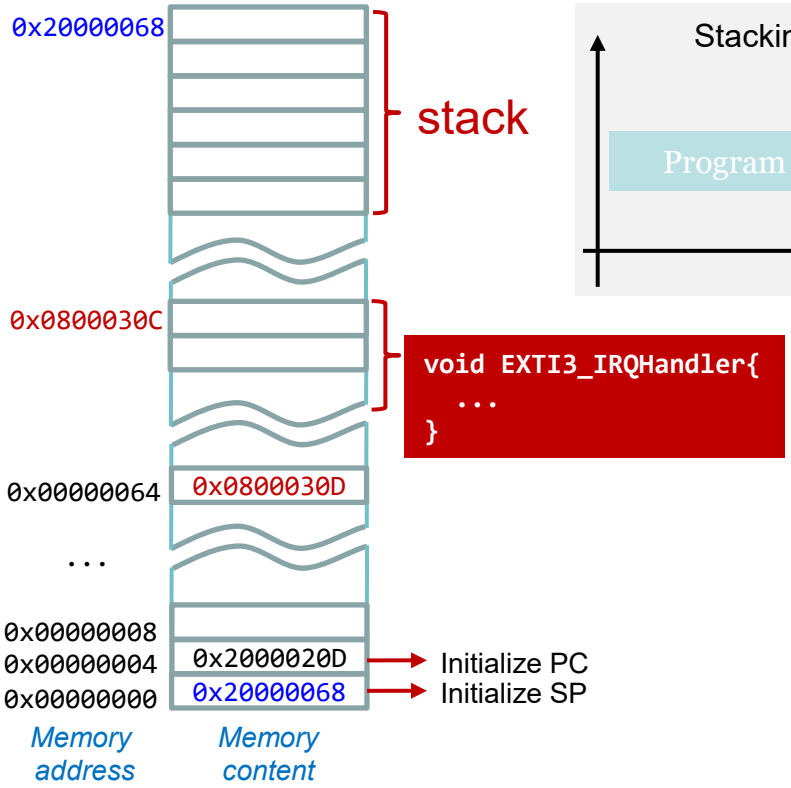


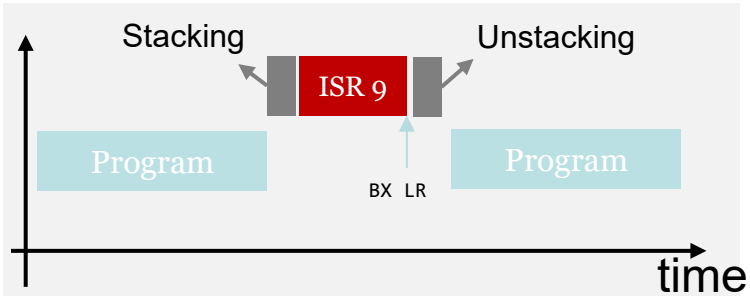


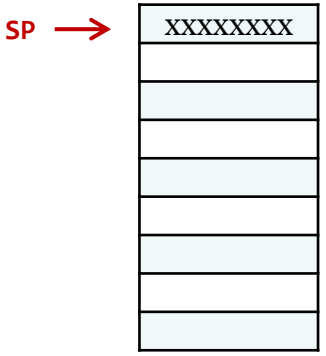


	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

Single Interrupt

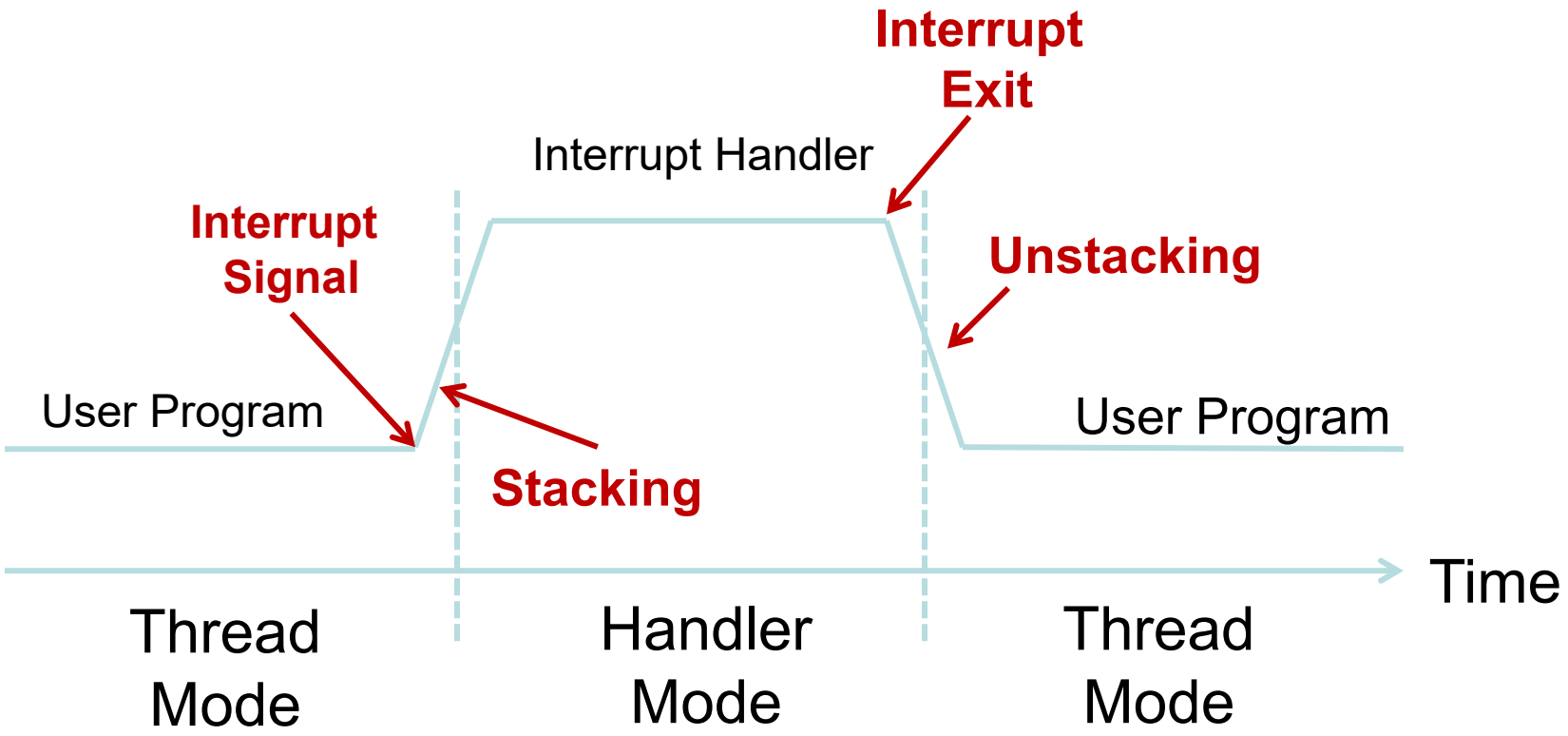






	EXTI3				
Interrupt Number	12	11	10	9	8
Enable Register	0	0	0	1	0
Active Register	0	0	0	0	0
Pending Register	0	0	0	0	0
Priority Register	3	4	7	2	8

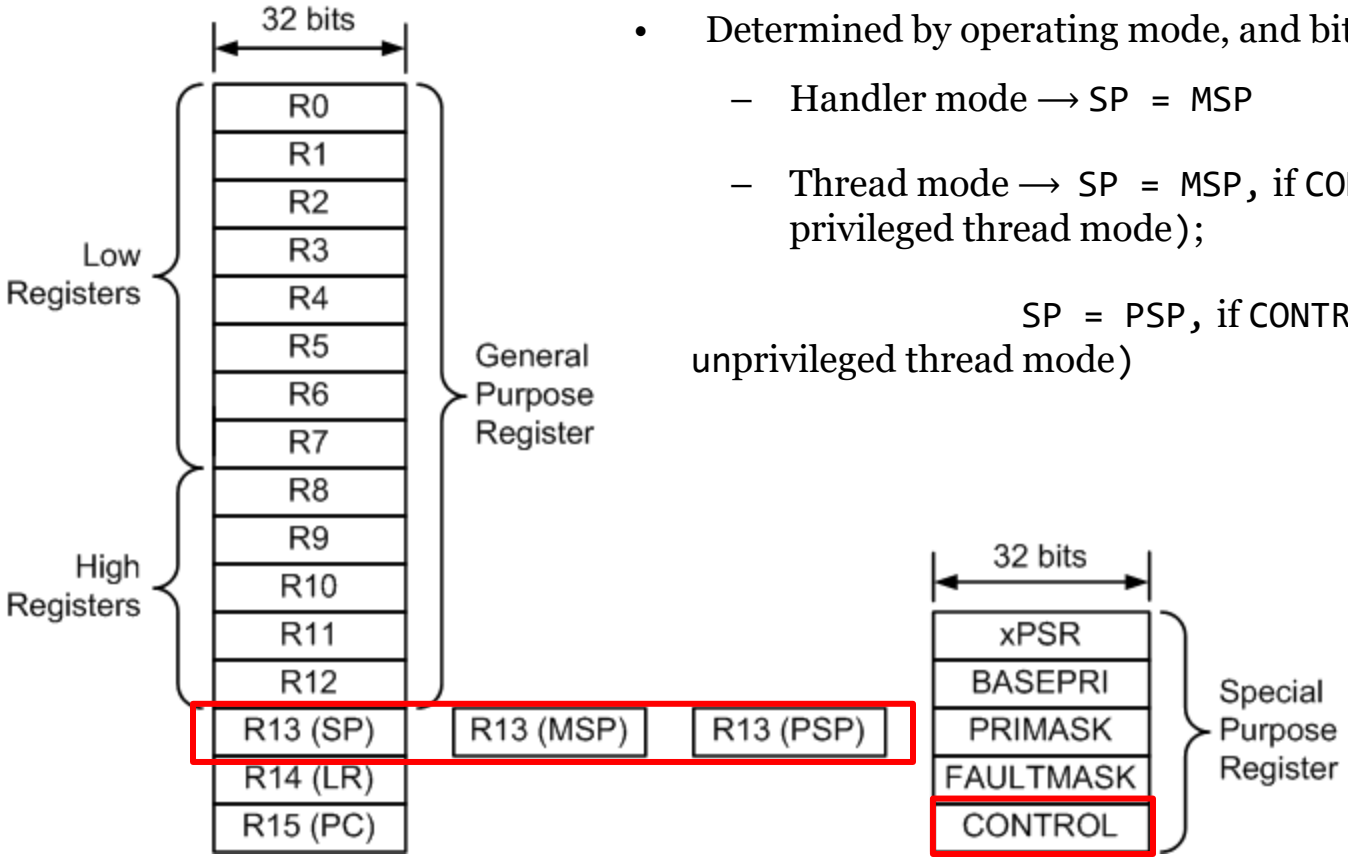
Stacking & Unstacking



Registers

- Two stack pointers: Main SP (MSP) and Process SP (PSP)
- Determined by operating mode, and bit 0 of the CONTROL register
  - Handler mode ⟶ `SP = MSP`
  - Thread mode ⟶ `SP = MSP`, if `CONTROL[0] = 0` (i.e., privileged thread mode);

    `SP = PSP`, if `CONTROL[0] = 1` (i.e., unprivileged thread mode)



MSP: Main Stack Pointer

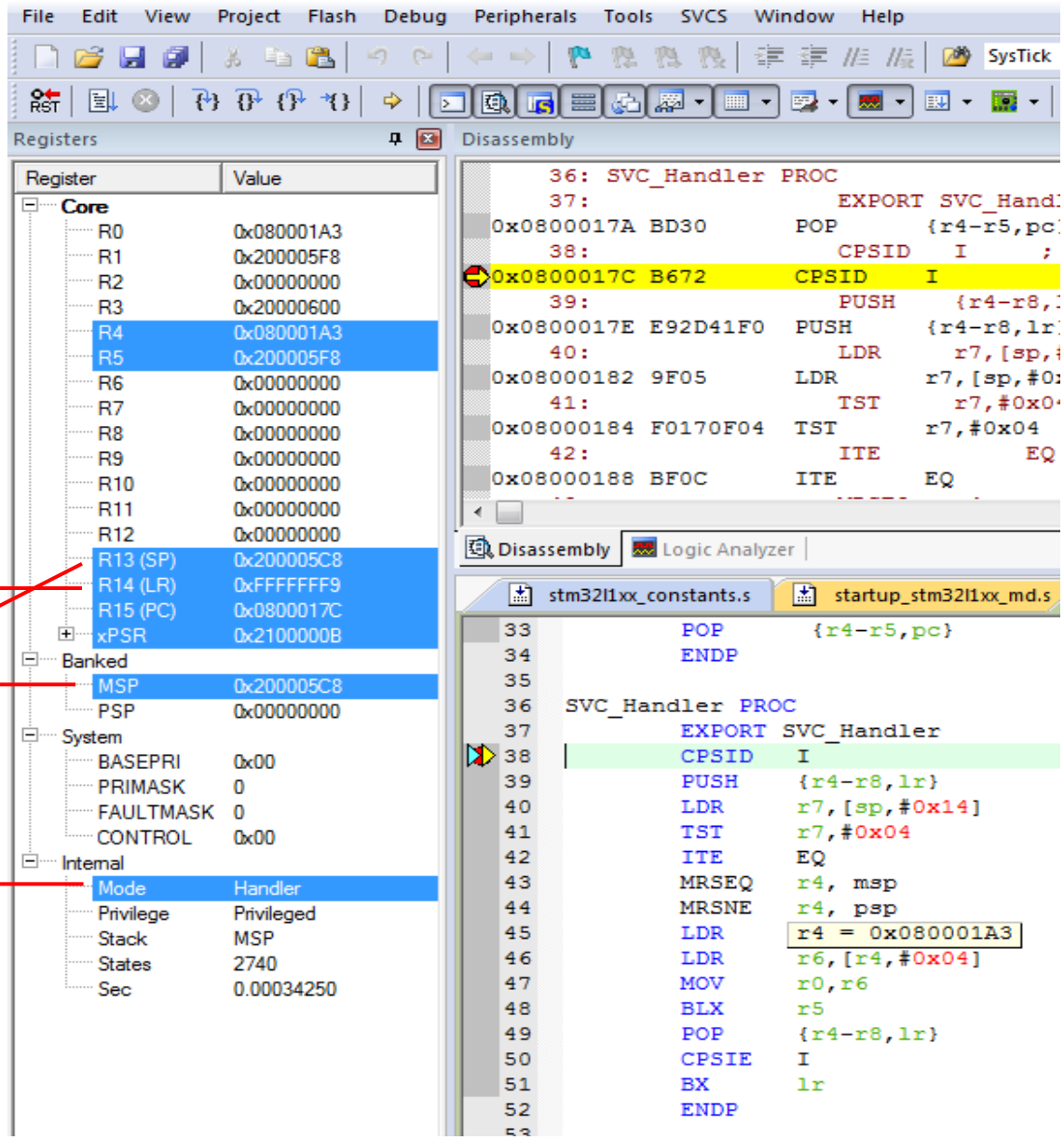
PSP: Process Stack Pointer

Register values in a interrupt service routine

LR = 0xFFFFFFF9

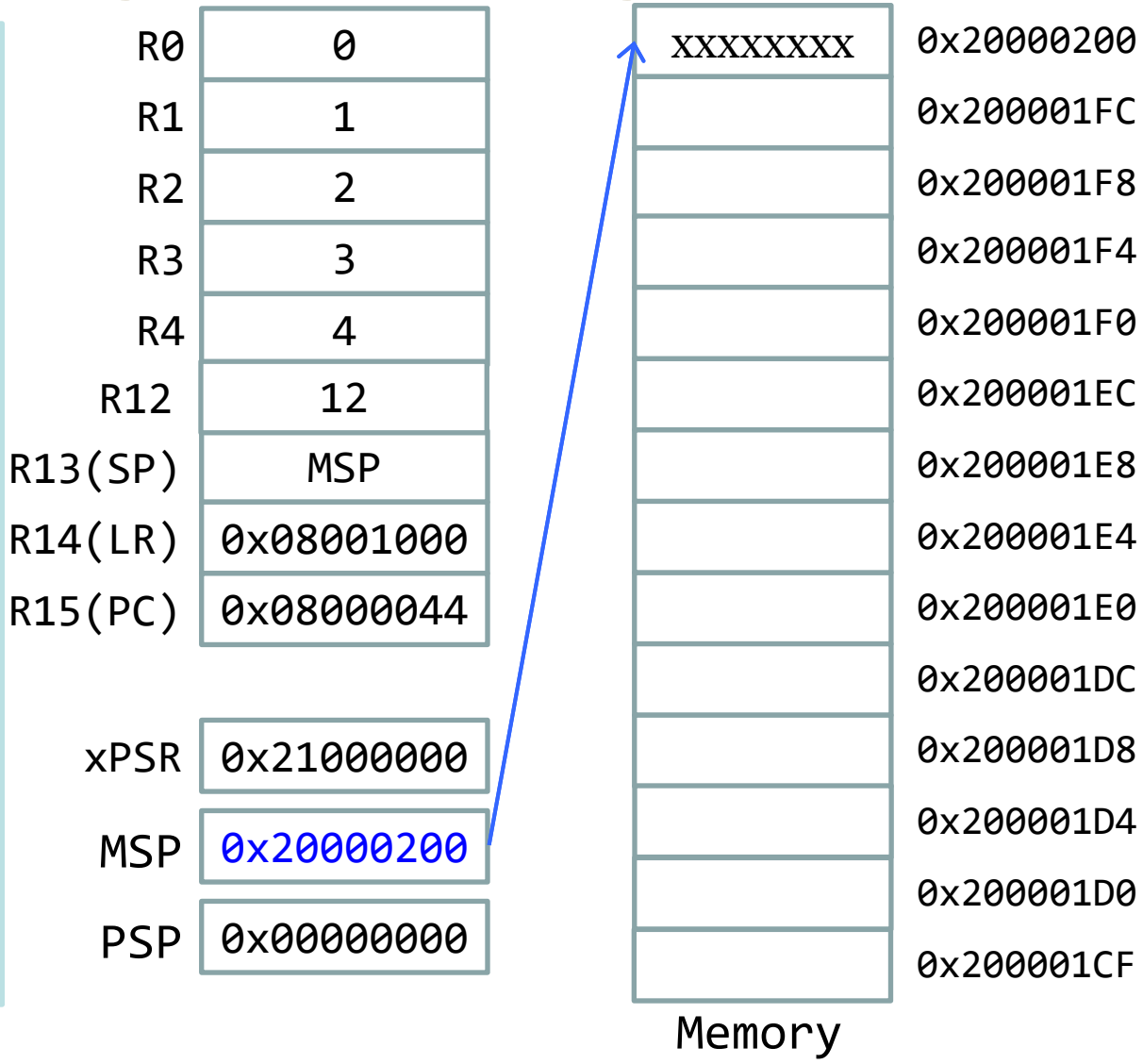
SP = MSP

ISR always in hander mode.



Interrupt: Stacking & Unstacking

```
__main PROC
                addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP          addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```



Suppose SysTick interrupt occurs when PC = 0x08000044

```
__main PROC
   ...
   MOV r3,#0
   ...
   ENDP

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

addr = 0x08000044

addr = 0x0800001C

R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0x08001000
R15(PC)	0x08000044
xPSR	0x21000000
MSP	0x20000200
PSP	0x00000000

	0x200001FC
	0x200001F8
	0x200001F4
	0x200001F0
	0x200001EC
	0x200001E8
	0x200001E4
	0x200001E0
	0x200001DC
	0x200001D8
	0x200001D4
	0x200001D0
	0x200001CF

Memory

```
__main PROC
          addr = 0x08000044
  …
  MOV r3,#0
  …
  ENDP      addr = 0x0800001C

SysTick_Handler PROC
  EXPORT SysTick_Handler
  ADD r3, #1
  ADD r4, #1
  BX  lr
  ENDP
```

STACKING

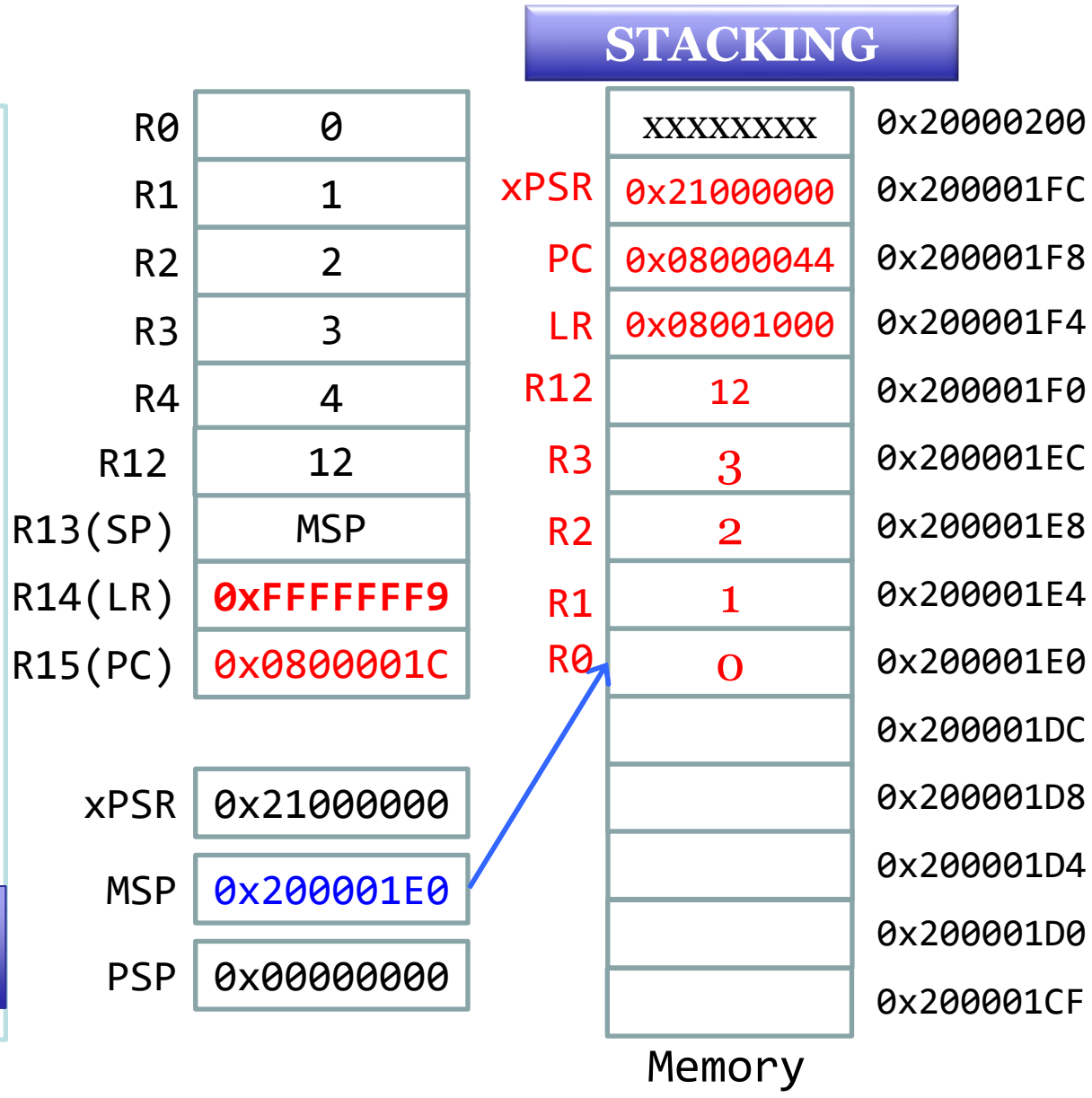
R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0xFFFFFFF9
R15(PC)	0x0800001C
xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

	xxxxxxxx	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x08000044	0x200001F8
LR	0x08001000	0x200001F4
R12	12	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

Memory

```
__main PROC
          addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP      addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

LR = 0xFFFFFFF9 to indicate MSP is used.



Interrupt: Stacking & Unstacking

```
__main PROC
                addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP          addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

R0	0
R1	1
R2	2
R3	4
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0xFFFFFFF9
R15(PC)	0x0800001C

xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

	XXXXXXXX	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x08000044	0x200001F8
LR	0x08001000	0x200001F4
R12	12	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

Memory

Interrupt: Stacking & Unstacking

```
__main PROC
                addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP          addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

R0	0
R1	1
R2	2
R3	4
R4	5
R12	12
R13(SP)	MSP
R14(LR)	0xFFFFFFF9
R15(PC)	0x08000020

xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

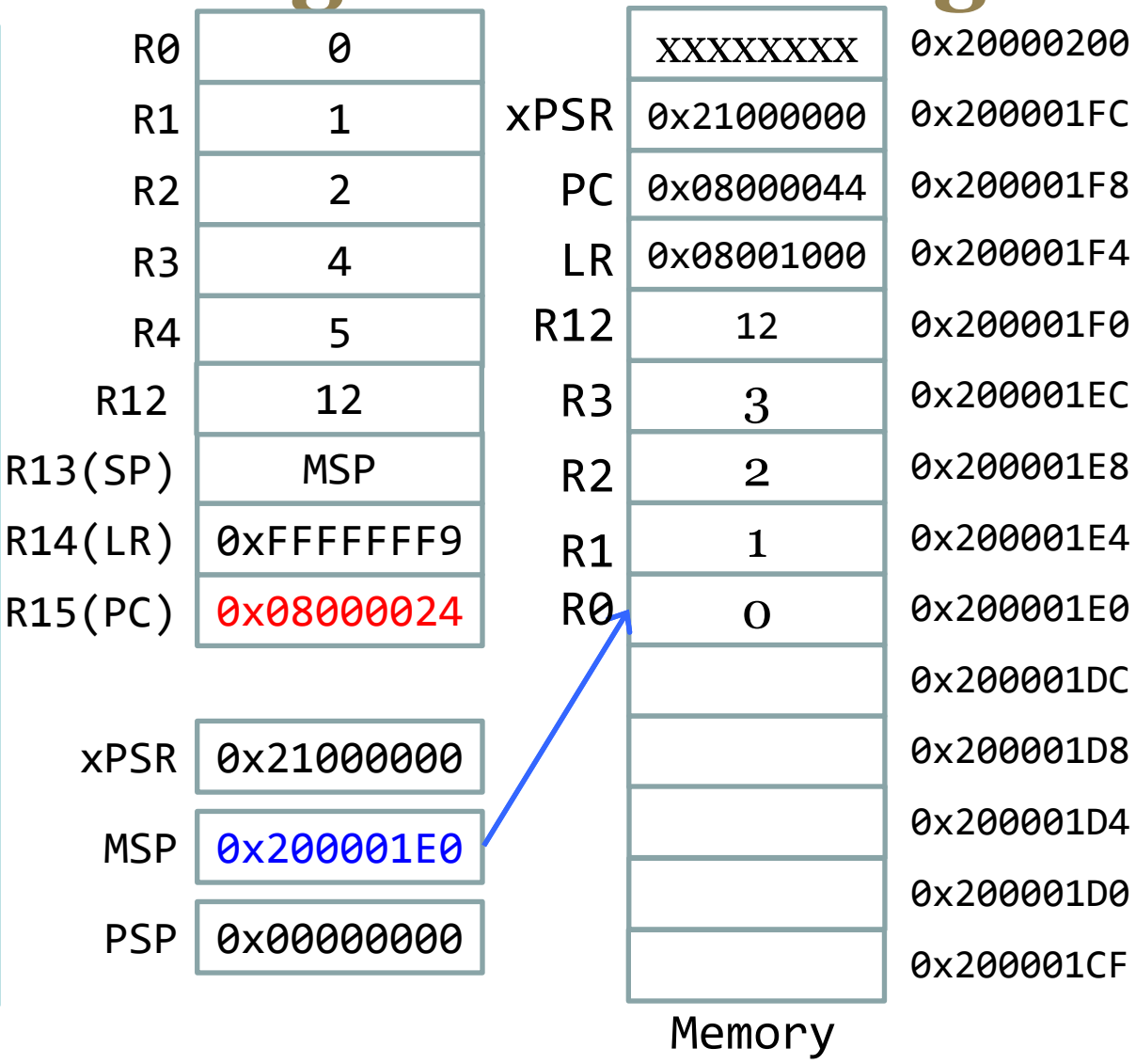
	XXXXXXXX	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x08000044	0x200001F8
LR	0x08001000	0x200001F4
R12	12	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

Memory

Interrupt: Stacking & Unstacking

```
__main PROC
                addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP          addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

LR = oxFFFFFFF9 to indicate MSP is used.



UNSTACKING

```
__main PROC
          addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP      addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

LR = oxFFFFFFF9 to indicate MSP is used.

R0	0
R1	1
R2	2
R3	4
R4	5
R12	12
R13(SP)	MSP
R14(LR)	0xFFFFFFF9
R15(PC)	0x08000024

xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

	XXXXXXXX	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x08000044	0x200001F8
LR	0x08001000	0x200001F4
R12	12	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

Memory

```
__main PROC
          addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP      addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

Note the new value of R3 is lost!!!

R0	0
R1	1
R2	2
R3	3
R4	5
R12	12
R13(SP)	MSP
R14(LR)	0x08001000
R15(PC)	0x08000044
xPSR	0x21000000
MSP	0x20000200
PSP	0x00000000

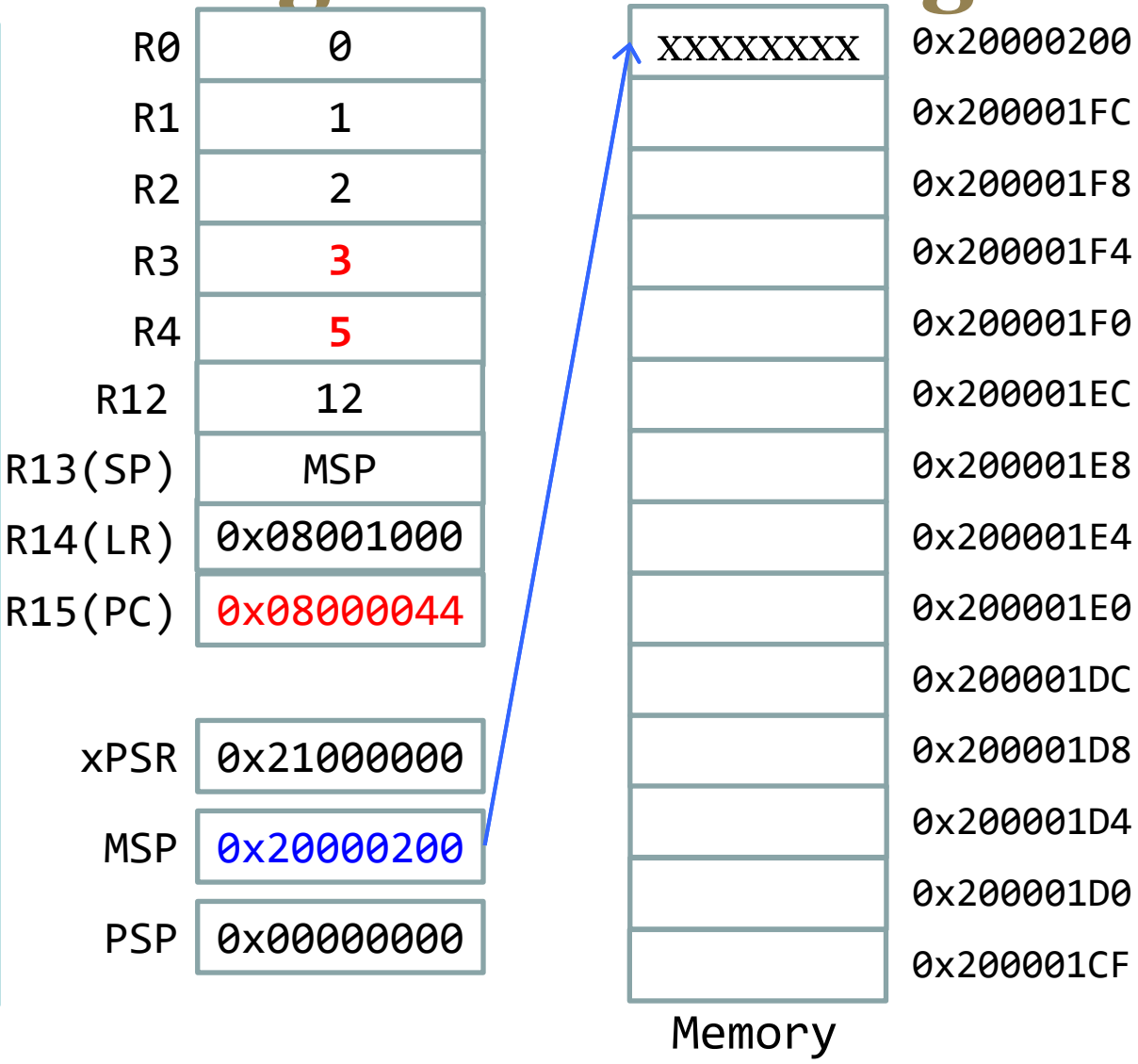
XXXXXXXX	0x20000200
	0x200001FC
	0x200001F8
	0x200001F4
	0x200001F0
	0x200001EC
	0x200001E8
	0x200001E4
	0x200001E0
	0x200001DC
	0x200001D8
	0x200001D4
	0x200001D0
	0x200001CF

Memory

Interrupt: Stacking & Unstacking

```
__main PROC
                addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP          addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r3, #1
   ADD r4, #1
   BX  lr
   ENDP
```

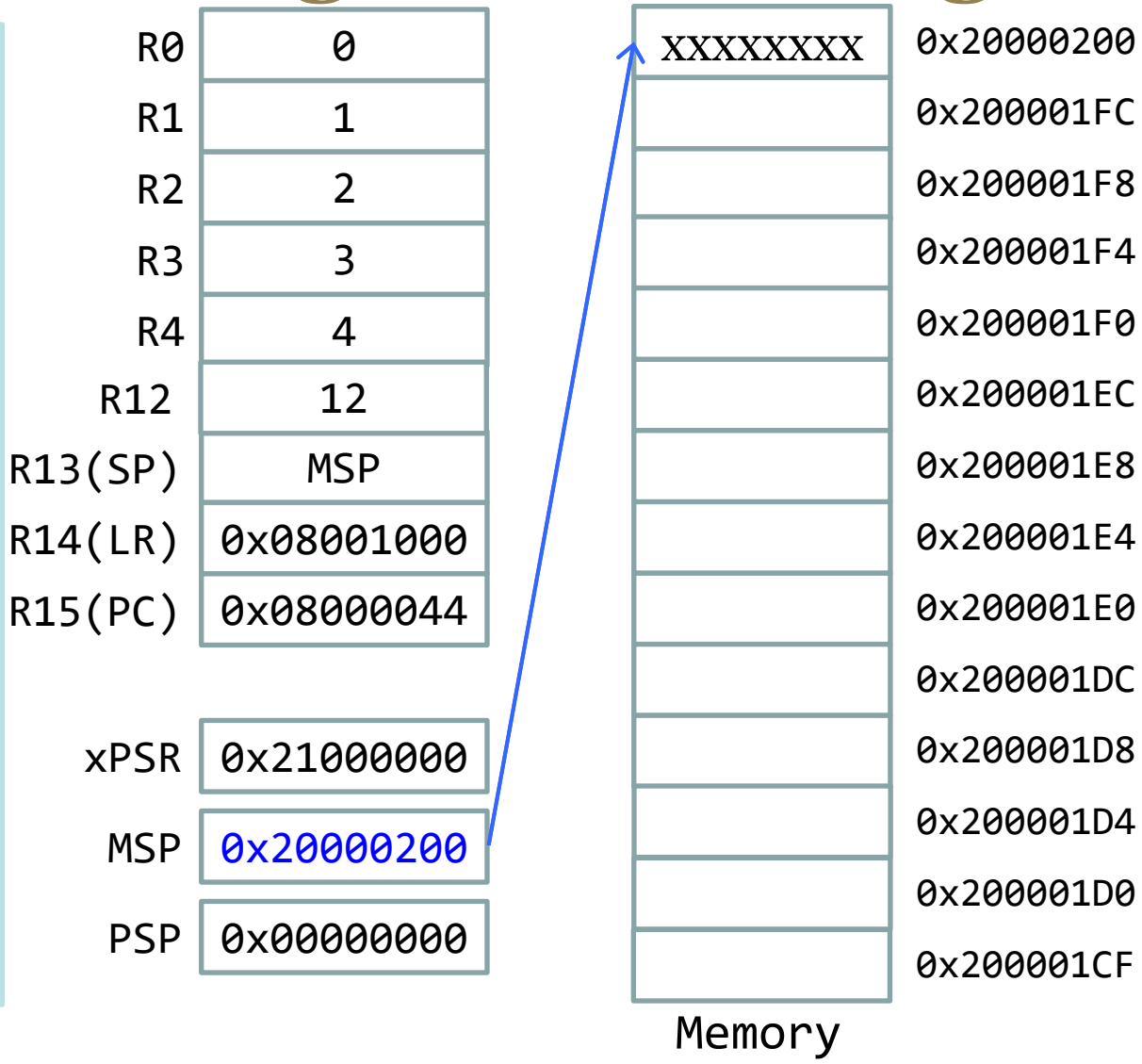
The Main program resumes!!!



What if ISR calls subroutine?

Interrupt: Stacking & Unstacking

```
__main PROC
                addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP          addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r4, #1
   BL  sine
   BX  lr
   ENDP
```



```
__main PROC
          addr = 0x08000044
  …
  MOV r3,#0
  …
  ENDP      addr = 0x0800001C

SysTick_Handler PROC
  EXPORT SysTick_Handler
  ADD r4, #1
  BL  sine
  BX  lr
  ENDP
```

LR = oxFFFFFFF9 to indicate MSP is used.

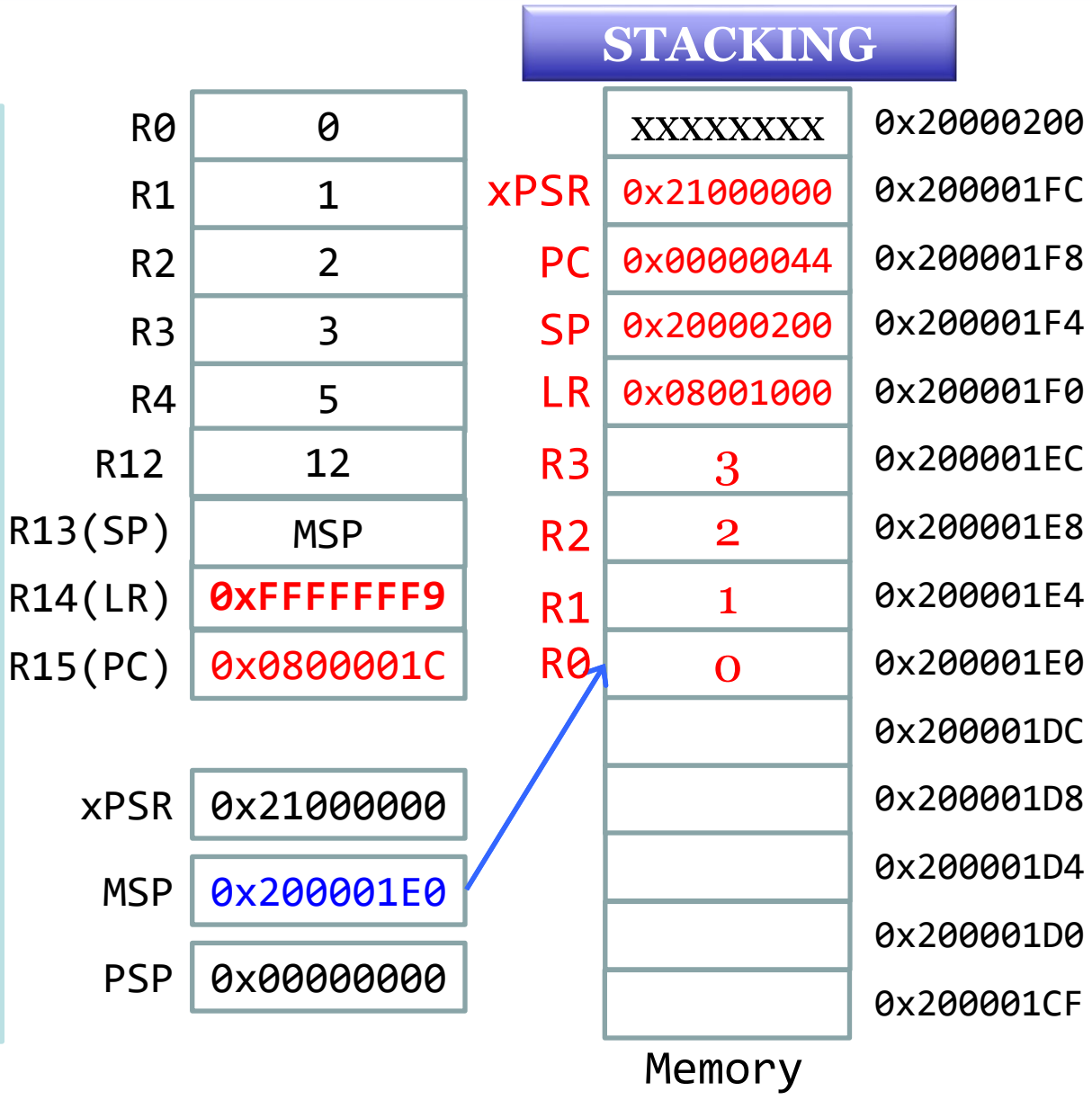
STACKING

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0xFFFFFFF9
R15(PC)	0x0800001C
xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

	XXXXXXXX	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x00000044	0x200001F8
SP	0x20000200	0x200001F4
LR	0x08001000	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

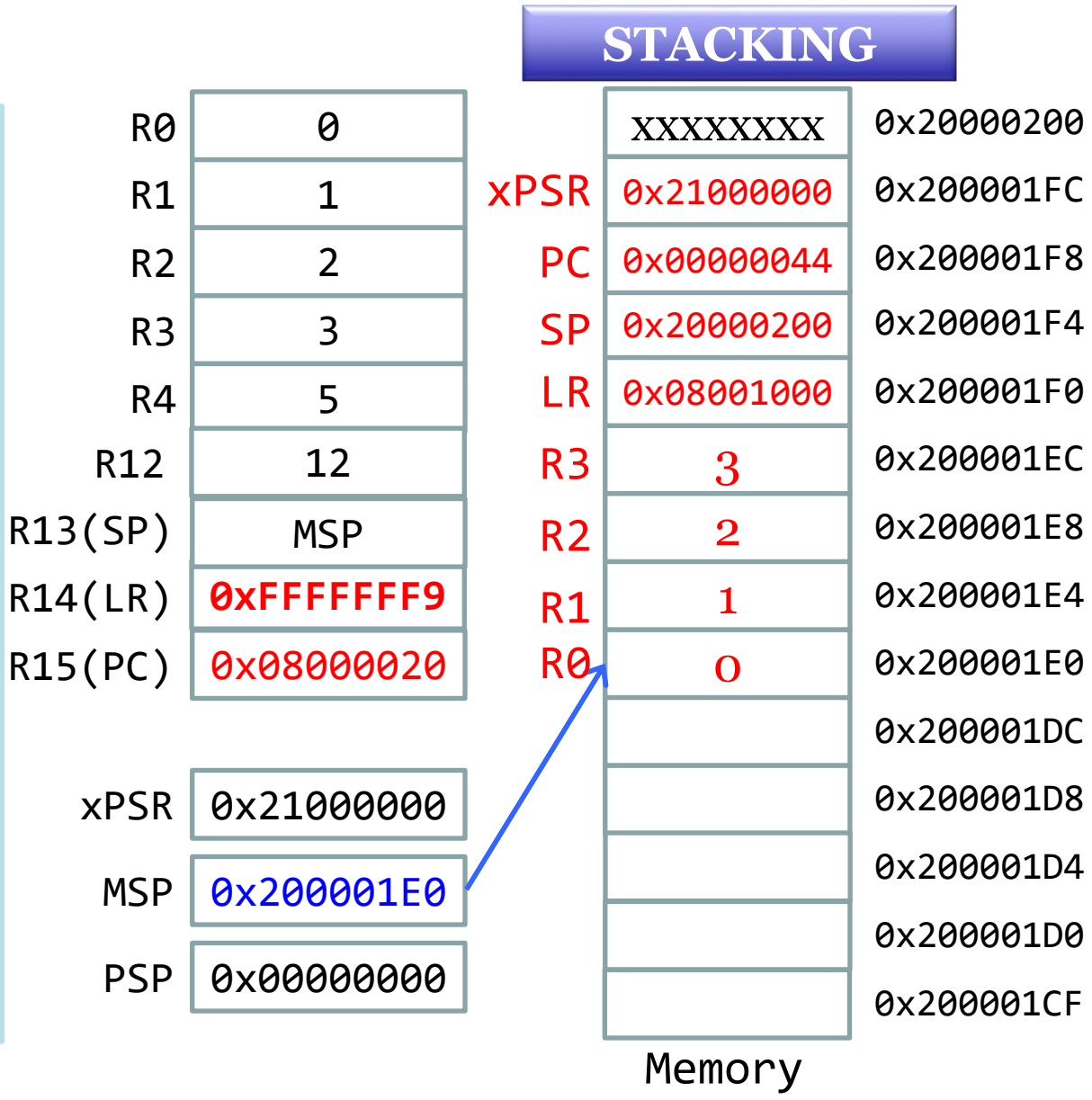
Memory

```
__main PROC
                addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP          addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r4, #1
   BL  sine
   BX  lr
   ENDP
```



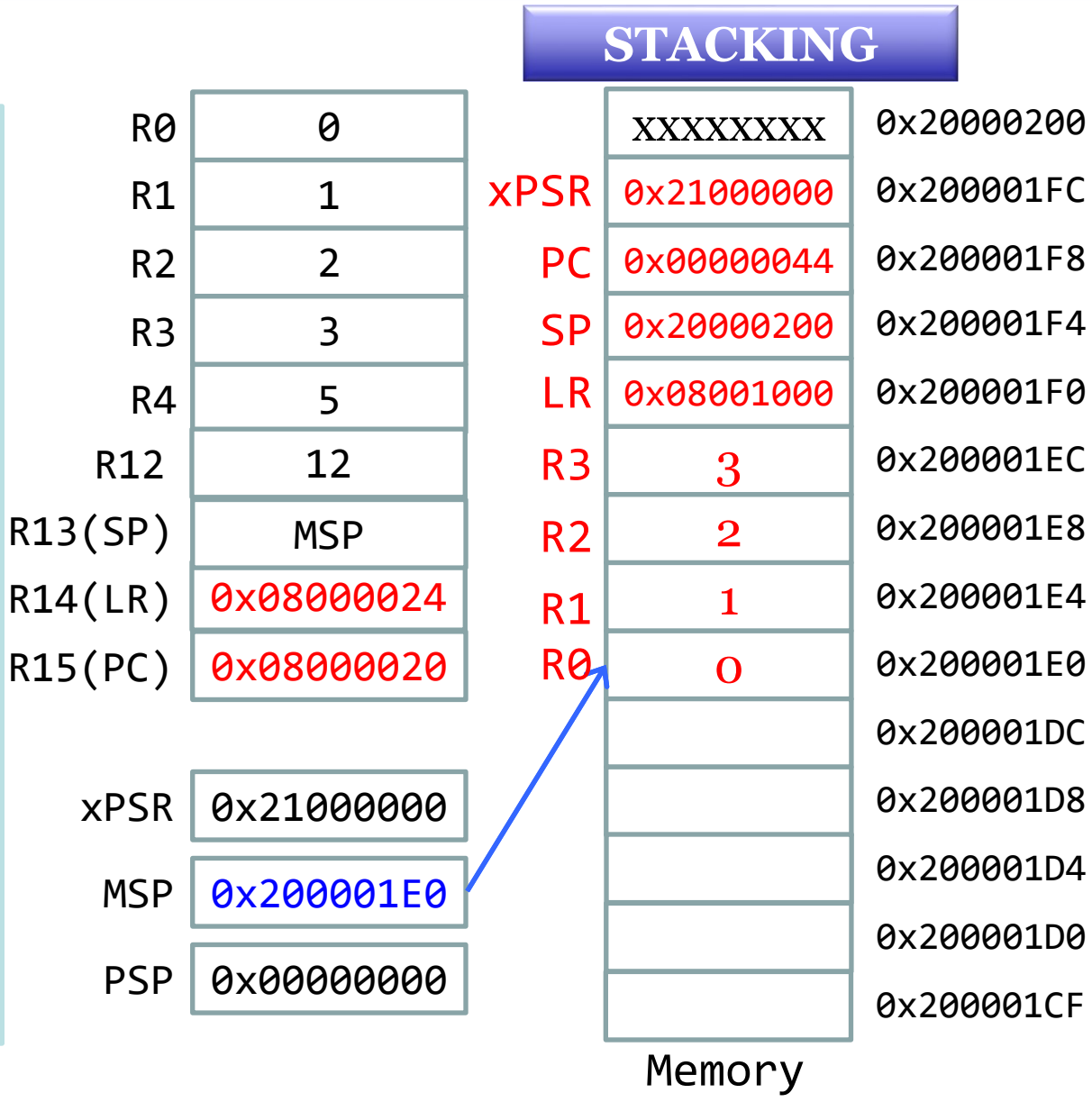
```
__main PROC
          addr = 0x08000044
  …
  MOV r3,#0
  …
  ENDP      addr = 0x0800001C

SysTick_Handler PROC
  EXPORT SysTick_Handler
  ADD r4, #1
  BL  sine
  BX  lr
  ENDP
```

BL sine
Updates LR register



```
__main PROC
          addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP      addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r4, #1
   BL  sine
   BX  lr
   ENDP
```

BL sine
Updates LR register



UNSTACKING won't occurs!

```
__main PROC
          addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP      addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   ADD r4, #1
   BL  sine
   BX  lr
   ENDP
```

BL sine
Updates LR register

R0	0
R1	1
R2	2
R3	3
R4	5
R12	12
R13(SP)	MSP
R14(LR)	0x08000024
R15(PC)	0x08000024
xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

	XXXXXXXX	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x00000044	0x200001F8
SP	0x20000200	0x200001F4
LR	0x08001000	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

Memory

Interrupt: Stacking & Unstacking

```
__main PROC
                addr = 0x08000044
   …
   MOV r3,#0
   …
   ENDP          addr = 0x0800001C

SysTick_Handler PROC
   EXPORT SysTick_Handler
   PUSH {lr}
   ADD r4, #1
   BL  sine
   POP {lr}
   BX  lr
   ENDP
```

R0	0
R1	1
R2	2
R3	3
R4	4
R12	12
R13(SP)	MSP
R14(LR)	0xFFFFFFF9
R15(PC)	0x0800001C
xPSR	0x21000000
MSP	0x200001E0
PSP	0x00000000

	XXXXXXXX	0x20000200
xPSR	0x21000000	0x200001FC
PC	0x00000044	0x200001F8
SP	0x20000200	0x200001F4
LR	0x08001000	0x200001F0
R3	3	0x200001EC
R2	2	0x200001E8
R1	1	0x200001E4
R0	0	0x200001E0
		0x200001DC
		0x200001D8
		0x200001D4
		0x200001D0
		0x200001CF

Memory

Fix the bug!